LAB 1 · FIRMWARE DEVELOPMENT WITH ESP IDF

Hello, ESP32-C61!

Programming an ESP32-C61 board with ESP-IDF

Prepared by Pragyan Sen Deka , Electronics Engineer, Associate , IIM Calcutta Innovation Park Technology and Innovation Council

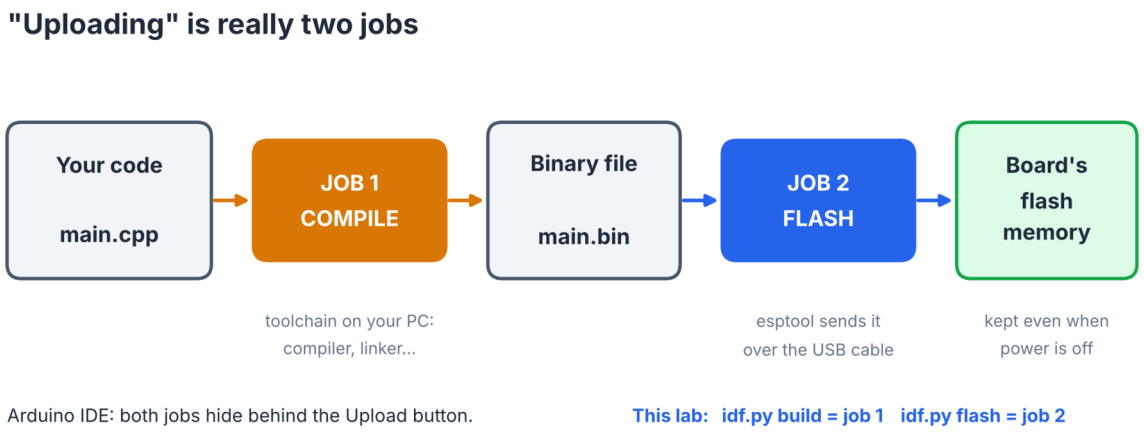


Figure 1. *The whole lab in one picture: we compile our code into a binary, then flash it onto the board.*

What	Used in this manual
Board	ESP32-C61 development board (with a CP210x USB-to-UART chip) (any other board can be used as well)
Framework	ESP-IDF **v5.5.5** (Espressif IoT Development Framework)
Arduino core	arduino-esp32 **3.3.12**, used as an ESP-IDF component
Computer	Windows 10 or 11 (64-bit), internet for the first build
Time needed	2–3 hours the first time (most of it is downloading and the first build)
Other boards	The same steps work for other ESP chips, including the ESP32-C2, which the Arduino IDE also can't program today. See Chapter 6.
Prior knowledge	None. If you can open a folder on Windows, you can do this lab.

INSIDE THIS MANUAL

Contents

Click any line to jump to it. In Word you can also open **View → Navigation Pane** to see every heading in a sidebar.

START HERE **Before you begin**

CHAPTER 1 **The big picture**

CHAPTER 2 **Layers of software, and why the Arduino IDE can't help (yet)**

CHAPTER 3 **Installing ESP-IDF v5.5.5**

CHAPTER 4 **Terminal survival kit**

CHAPTER 5 **The lab: Hello world in 8 steps**

- Step 1: Open the ESP-IDF terminal and check the version
- Step 2: Create the project
- Step 3: Explore the project files
- Step 4: Pin the Arduino core version
- Step 5: Set the target chip
- Step 6: Build
- Step 7: Find your board's port
- Step 8: Flash and monitor

CHAPTER 6 **Using this method for other ESP chips**

CHAPTER 7 **Troubleshooting**

CHAPTER 8 **Quick reference**

CHAPTER 9 **Glossary**

CHAPTER 10 **Check your understanding**

CHAPTER 11 **What's next**

START HERE

Before you begin

In this lab you will make an ESP32-C61 board print **Hello world!** on your computer screen once every second. That sounds small, but to get there you will meet the same tools that professional embedded engineers use every day, and you will understand what each one does instead of just pressing a button and hoping.

What you will be able to do and explain

- Explain what *compiling* and *flashing* mean, and why uploading a program is really two separate jobs.
- Explain why the Arduino IDE cannot program the ESP32-C61 today, and how we get around it.
- Install the right version of ESP-IDF and use its terminal with confidence.
- Create, configure and build an ESP-IDF project that uses the Arduino core.
- Read the build output: hexadecimal sizes, partitions and the files that get produced.
- Explain how the board starts up and how esptool writes your program into flash memory.
- Find and fix the most common problems on your own.

What you need

Item	Details
ESP32-C61 board	Any ESP32-C61 development board. This manual uses one with a CP210x USB-to-UART chip.
USB cable	It must be a *data* cable. Some cheap cables only carry power for charging and will not work.
Windows PC	Windows 10 or 11, about 10 GB of free disk space, and internet access for installing and the first build.
ESP-IDF v5.5.5	Chapter 3 shows how to install it. Exactly this version family (5.5) matters.
Patience	The very first build takes 5–15 minutes. After that, builds take seconds.

How to read this manual

Every new word is explained the first time it appears, usually in a grey **New words** box. Things you type look like `idf.py build`. Longer commands and screen output appear in dark boxes:

TYPE THIS

```
idf.py --version
```

```
YOU SHOULD SEE (similar to this)
ESP-IDF v5.5.5
```

You will also find these coloured boxes:

Box	What it is for
Why are we doing this?	The reason behind a step. Read these: they are what makes you able to fix things later.
Think of it like this	An everyday comparison (an *analogy*) for a tricky idea.
Common mistake	Something students often get wrong, and how to avoid it.
Fun fact	Extra curiosity. Nice to know, not needed for the lab.
Try it	A small optional experiment to see something for yourself.
Checkpoint	A quick question to test yourself before moving on. Answers are in Chapter 10.

> NOTE
>
> **Not using an ESP32-C61?** This manual uses the C61 as its example, but the method works for any ESP chip that ESP-IDF supports. For a board the Arduino IDE can't program (such as the ESP32-C2), follow the same steps and change one word in Step 5. Chapter 6 explains exactly what changes for each chip.

> NOTE
>
> Chapters 1 to 4 explain the ideas and set up your computer. Chapter 5 is the hands-on lab, in 8 steps. If your computer is already set up, you can still read Chapters 1, 2 and 4 first: the lab makes much more sense afterwards.

CHAPTER 1

The big picture

1.1 Meet your board

Your ESP32-C61 board is a small circuit board built around a **microcontroller**: a complete little computer squeezed onto one chip. It has a processor that runs instructions, a small amount of working memory, and **pins** that you can connect to LEDs, buttons and sensors. Microcontrollers are everywhere: in washing machines, TV remotes, smart bulbs and car keys.

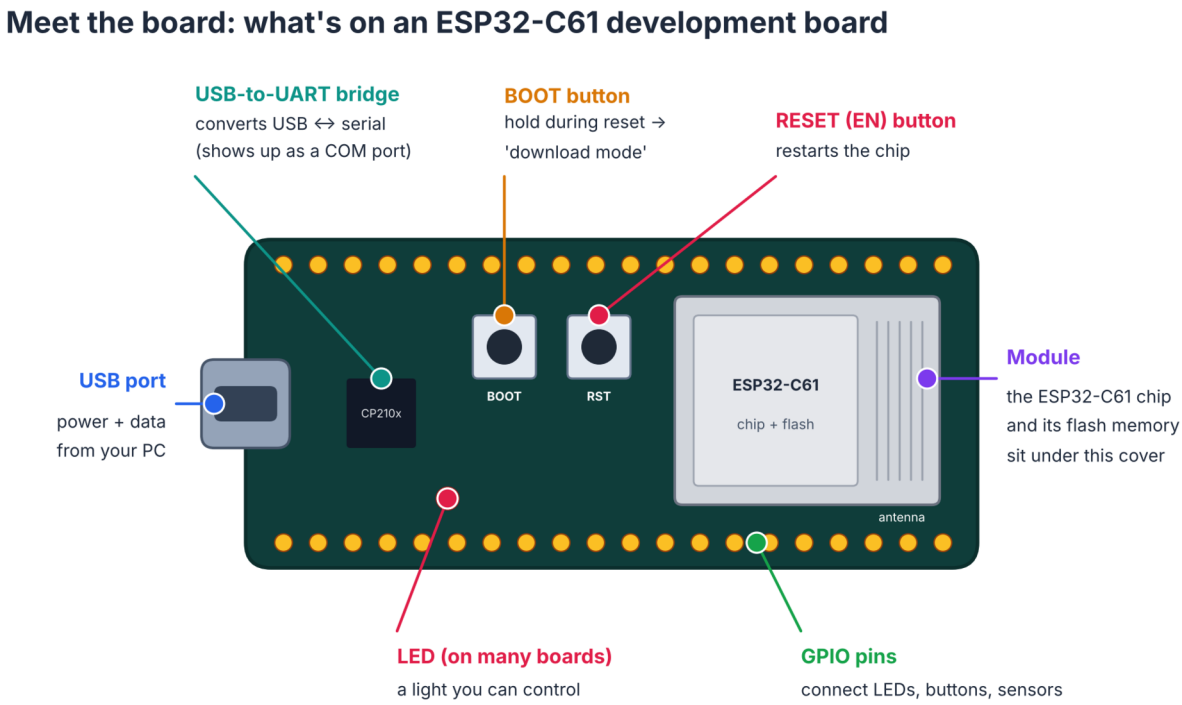


Figure 2. *The main parts of a typical ESP32-C61 development board. Your board may look slightly different, but it has the same parts.*

Inside the module, next to the chip, is a **flash memory** chip. Flash memory keeps what is written in it even when the power is switched off, just like a pen drive. This is where your program will live. Every time the board powers on, it reads your program from flash and runs it.

ESP32-C61 at a glance	Value	What it means for you
Processor	1 core, RISC-V, up to 160 MHz	Runs about 160 million simple steps per second, one at a time.
Working memory (SRAM)	320 KB	Where variables live while the program runs. Cleared at power-off.
Wireless	Wi-Fi 6 (2.4 GHz) and Bluetooth LE 5	Can join Wi-Fi and talk to phones over Bluetooth Low Energy.
Program memory (flash)	On the module (our build assumes 2 MB)	Where your program is stored. Kept at power-off.

FUN FACT

A typical phone has about 8 GB of RAM. That is roughly **25,000 times** the 320 KB in your ESP32-C61. Microcontroller programmers learn to do a lot with very little, which is part of the fun.

NEW WORDS IN THIS SECTION

Microcontroller: a small computer on a single chip, made to control hardware.

Pin / GPIO: a metal leg of the chip you can connect things to. GPIO means General-Purpose Input/Output: a pin your program can switch on and off, or read.

Flash memory: memory that keeps its contents with the power off. Holds your program.

RAM (SRAM): fast working memory for running programs. Loses everything when power is off.

1.2 The chip only understands numbers

You write programs in C++, using English-like words such as `digitalWrite` and `delay`. The processor cannot read any of that. It only understands **machine code**: long lists of numbers, where each number means one tiny instruction such as “add these two values” or “jump back to that step”.

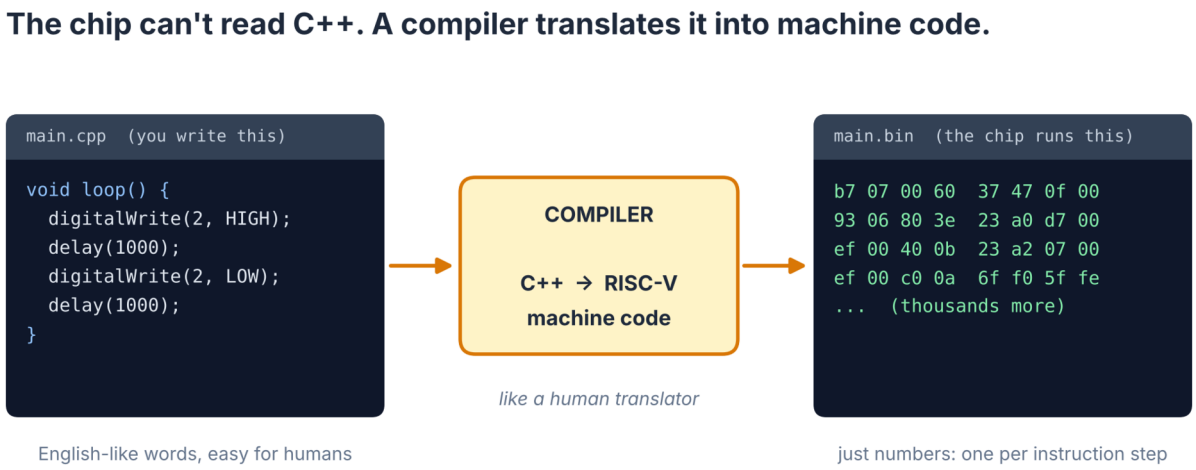


Figure 3. *A compiler translates your C++ code into machine code. (The machine-code numbers here are only an illustration.)*

The program that does this translation is called a **compiler**. The compiler is only one tool in a set of tools that work together to turn code into something the chip can run; that whole set is called the **toolchain**. The final product is a **binary** file (ending in `.bin`) that holds exactly the bytes to be written into flash memory.

THINK OF IT LIKE THIS

Imagine you wrote a letter in English for a friend who only reads Japanese. A translator rewrites it in Japanese. The compiler is that translator: you write in C++, it rewrites everything in the chip’s own language.

NEW WORDS IN THIS SECTION

Machine code: instructions as numbers: the only language a processor understands.

Compiler: a program that translates source code (like C++) into machine code.

Toolchain: the complete set of tools (compiler, linker and helpers) that turns code into a runnable binary.

Binary (.bin): a file containing raw machine code and data, ready to be written into flash.

Source code: the human-readable code that people write, before it is compiled.

1.3 “Uploading” is really two jobs

In the Arduino IDE you press one Upload button. Behind that button, two completely separate jobs happen:

1. **Compiling** (job 1): the toolchain on your computer turns your code into a binary file.
2. **Flashing** (job 2): a program called **esptool** sends that binary through the USB cable and writes it into the board’s flash memory.

In this lab we do each job ourselves, with commands we can see and understand: `idf.py build` for job 1 and `idf.py flash` for job 2 (Figure 1 on the cover shows both).

CHECKPOINT

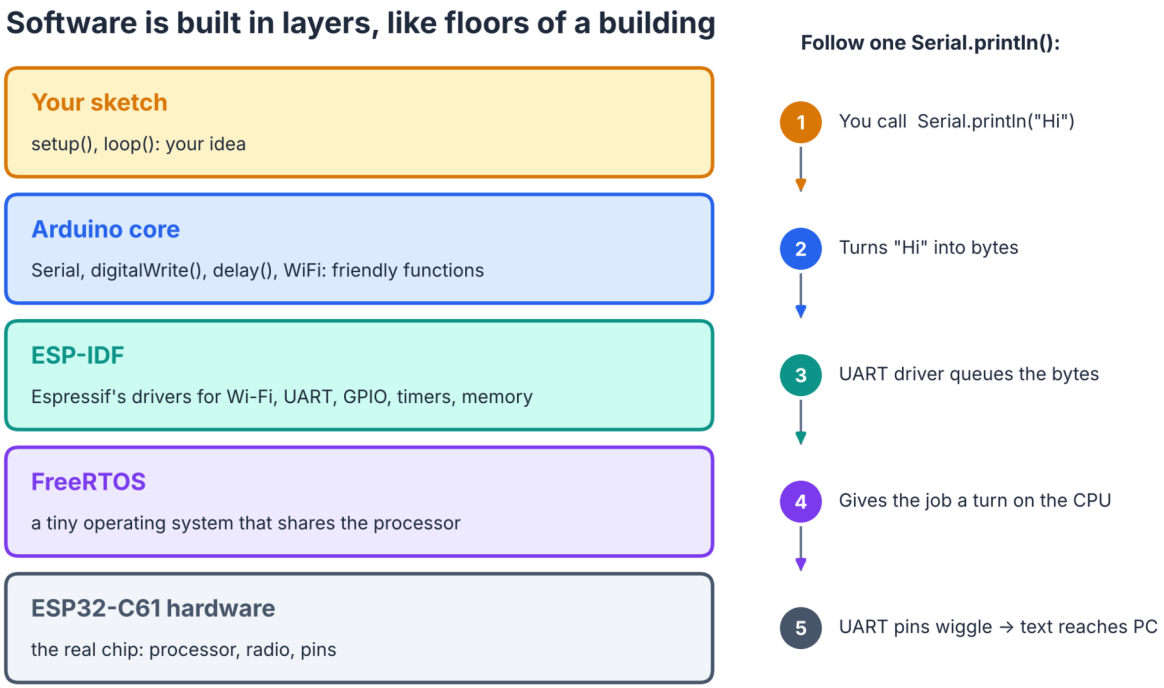
Your friend says: “my code compiled fine, so it must be on the board now.” What is wrong with that idea?

CHAPTER 2

Layers of software, and why the Arduino IDE can’t help (yet)

2.1 Software is built in layers

When you call `Serial.println("Hi")`, you are using code that thousands of other people wrote. That code is organised in layers, like the floors of a building. Each floor rests on the floors below it.



Each layer builds on the ones below it, so your sketch never has to control the hardware directly.

Figure 4. *The software layers in this lab, and the journey of one Serial.println() call from your sketch down to the hardware.*

- **Your sketch** is your own code: `setup()` and `loop()`.
- The **Arduino core** is the code that makes Arduino functions like `Serial`, `digitalWrite()` and `delay()` work on a particular family of chips. For ESP32 chips, Espressif (the company that makes them) maintains it.
- **ESP-IDF** (Espressif IoT Development Framework) is Espressif’s official system for programming their chips. It knows how to drive the Wi-Fi radio, the serial port, the pins, the timers and the memory. The Arduino core for ESP32 is not built from scratch: it is a friendly layer on top of ESP-IDF.
- **FreeRTOS** is a tiny operating system inside ESP-IDF that shares the processor between several jobs. You will meet it properly in Step 3.
- At the bottom is the **hardware**: the real ESP32-C61 chip.

NEW WORDS IN THIS SECTION

Arduino core: the code behind Arduino functions (Serial, delay, digitalWrite...) for one chip family.

Framework: a large set of ready-made code and tools that you build your own program on top of.

ESP-IDF: Espressif's official framework for ESP32 chips. Pronounced "E-S-P I-D-F".

Driver: code that knows how to operate one piece of hardware, such as the serial port. (Windows also has drivers: small programs that let it talk to a device you plug in.)

2.2 The mystery of the two errors

This manual exists because the Arduino IDE refused to program the ESP32-C61. It is worth understanding exactly why, because the reason explains everything we do afterwards.

Error 1: "This chip is ESP32-C61, not ESP32-C6"

With no C61 option in the board list, the closest choice was the ESP32-C6. The IDE compiled the code for the C6 and then tried to flash it. Before writing anything, **esptool** asked the chip what it was. Every Espressif chip stores an identity number inside it, and this one answered "I am an ESP32-C61". esptool stopped, because a program built for a different chip would not run correctly. That error was a safety check doing its job.

Error 2: a file that "does not exist"

The Arduino core actually already contains an ESP32-C61 board entry, but it is *hidden* in a settings file called `boards.txt`. When the entry was unhidden and the IDE tried to build, it failed with a missing-file error:

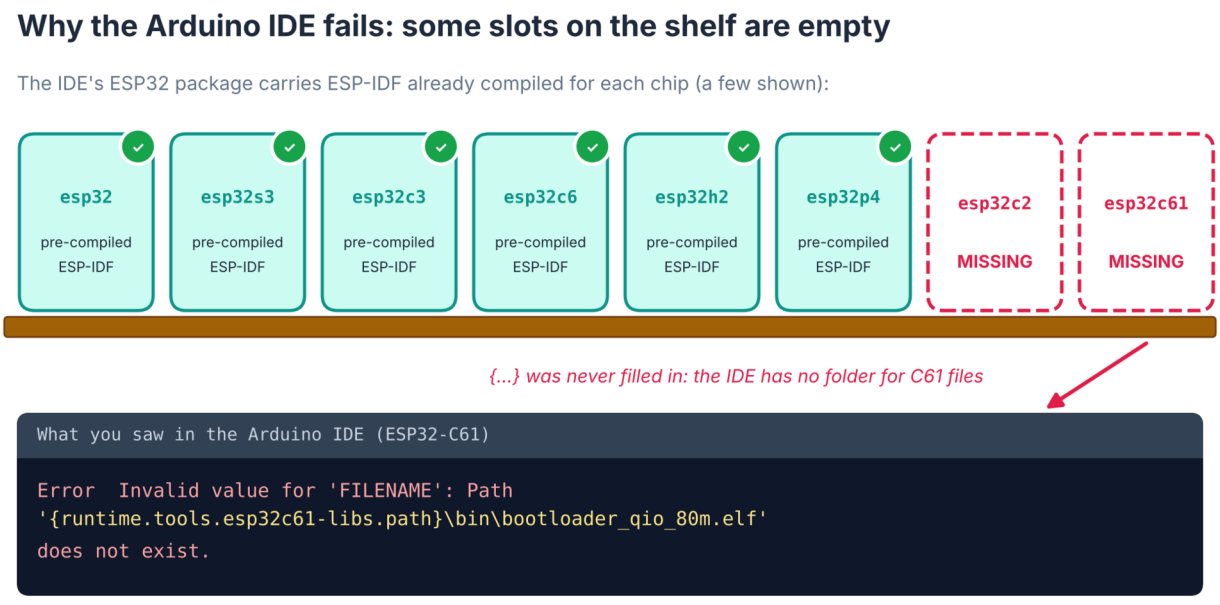


Figure 5. *The Arduino IDE expects ESP-IDF to be pre-compiled for each chip. For the ESP32-C61 (and the ESP32-C2) there is nothing on the shelf, so the path in the error is left as an unfilled {placeholder}.*

ESP-IDF is huge: compiling it takes many minutes. To save you that wait, the Arduino IDE's ESP32 package ships ESP-IDF **pre-compiled**, already translated into machine code, once for each supported chip. Espressif

has not shipped the pre-compiled ESP32-C61 version yet. The IDE went looking for those files and found nothing. That is also why the board entry was hidden: it cannot work until those files exist.

> NEW WORDS IN THIS SECTION
>
> **Pre-compiled:** already translated into machine code ahead of time, so it does not need compiling again.
>
> **esptool:** Espressif's program for talking to the chip over USB: identifying it, erasing and writing flash, resetting it.

2.3 Our plan: cook it ourselves

You may notice that the missing piece, ESP-IDF, is something we can install ourselves. ESP-IDF itself fully supports the ESP32-C61. So we turn the situation around: ESP-IDF becomes the main system, and the Arduino core is added into it as a **component**. ESP-IDF then compiles everything from **source code** (the original human-readable code) specifically for the C61: ESP-IDF itself, the Arduino core, and your sketch. Nothing needs to be pre-compiled.

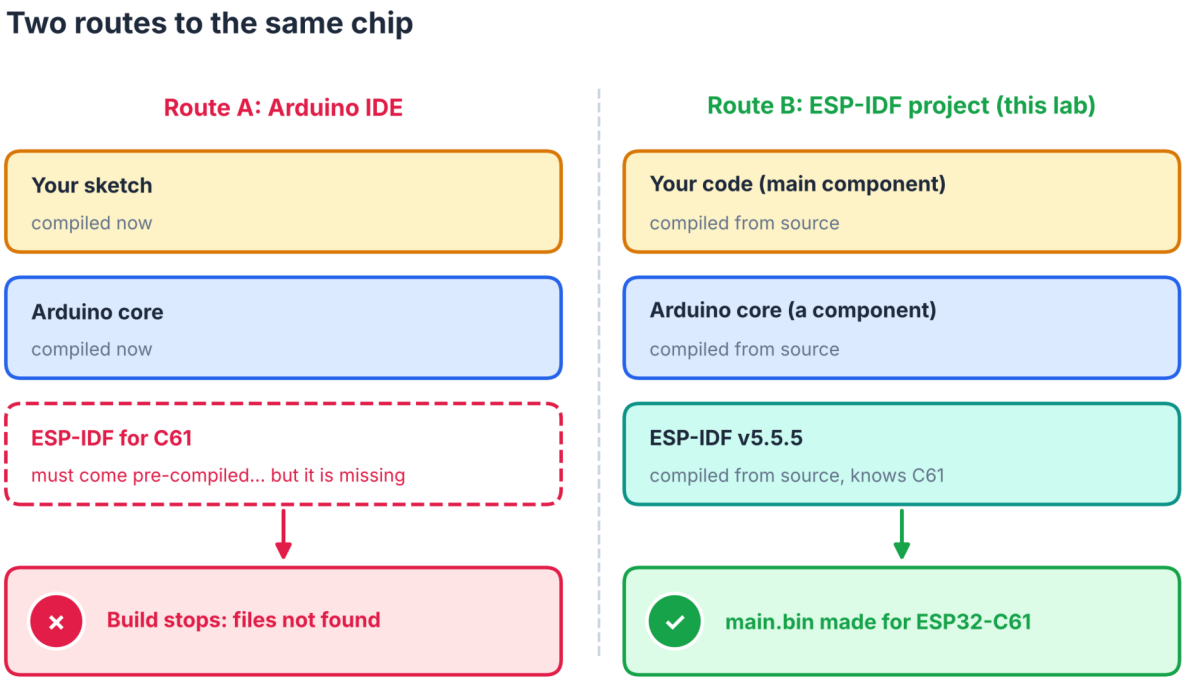


Figure 6. *Route A needs a pre-compiled piece that does not exist yet. Route B compiles everything from source, so nothing is missing.*

> THINK OF IT LIKE THIS
>
> The Arduino IDE is like a box of ready meals: quick to heat up, but if your dish is not in the box, you go hungry. Route B is cooking from raw ingredients in ESP-IDF's kitchen. It takes longer the first time (the first build is slow), but you can cook any dish, including the C61 one.

Your sketches stay Arduino code. Only the tools that build and flash them change.

NEW WORDS IN THIS SECTION

Component: in ESP-IDF, a self-contained package of code that a project can include. Your own code is a component too.

NOTE

This isn't only a C61 problem. Every time Espressif releases a new chip, ESP-IDF supports it months before the Arduino IDE ships pre-compiled files for it. Today the ESP32-C2 is in the same situation as the C61. Chapter 6 shows how to use this manual's method for any of them, and how to tell which route a board needs.

CHECKPOINT

In one sentence: why can ESP-IDF build for the C61 when the Arduino IDE can't?

CHAPTER 3

Installing ESP-IDF v5.5.5

3.1 Why the exact version matters

Software versions must fit together, just like a charger plug must fit its socket. The Arduino core version we use, **3.3.12**, was designed and tested with ESP-IDF **v5.5**. Newer is not automatically better here: ESP-IDF v6 changed many things, and the 3.3 Arduino core is not designed for it.

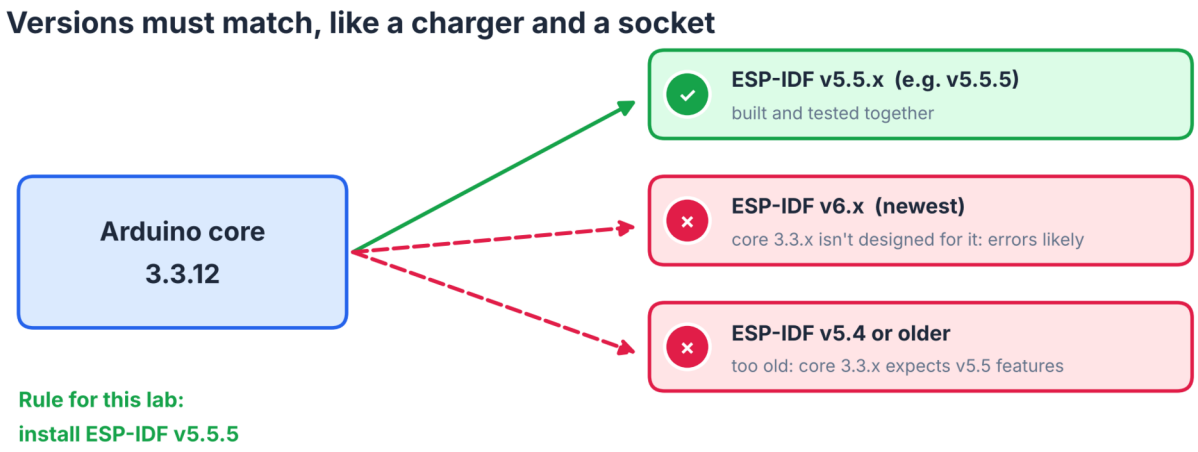


Figure 7. *Arduino core 3.3.12 fits ESP-IDF v5.5.x. Other ESP-IDF versions are likely to cause errors.*

Software	Version for this lab	Why
ESP-IDF	**v5.5.5** (latest bug-fix release of v5.5)	The version the Arduino core 3.3.x is built for.
Arduino core (arduino-esp32)	**3.3.12**	Downloaded automatically into your project in Step 5.
Python, CMake, Ninja, compiler	whatever the installer gives you	The ESP-IDF installer picks matching versions of these tools.

> COMMON MISTAKE
>
> Espressif's newest installer, the **ESP-IDF Installation Manager (EIM)**, offers an "easy" installation that installs the *newest* ESP-IDF (v6.x). That is the wrong version for this lab. Use one of the two options below, which both give you v5.5.5.

3.2 Option A (recommended): the ESP-IDF v5.5.5 offline installer

An *offline* installer contains everything inside one big download, so it does not need to download more pieces while installing. This makes it the most reliable choice for lab computers.

1. In your web browser, open **https://dl.espressif.com/dl/esp-idf/**

2. In the list of Windows offline installers, download **ESP-IDF v5.5.5** (about 1.6 GB).
3. Run the downloaded file and follow the screens. Keep the default install folder unless your teacher tells you otherwise. Whatever folder is used must be short and contain **no spaces** (see 3.3).
4. If the installer offers to install USB drivers, keep them selected. They let Windows talk to your board.
5. When it finishes, it adds Start-menu entries with names like **ESP-IDF 5.5 CMD** and **ESP-IDF 5.5 PowerShell**. These open the special ESP-IDF terminal you will use in Chapter 4.

3.3 Option B: the ESP-IDF Installation Manager (EIM)

If you or your college already use EIM, you can ask it for this exact version from the command line instead of using its “easy” mode:

TYPE THIS

```
winget install Espressif.EIM-CLI
eim install -i v5.5.5
```

`winget` is Windows’ built-in tool for installing programs from the command line; the first line installs EIM itself. The second line asks EIM to install ESP-IDF version 5.5.5 (`-i` stands for “IDF version”).

3.4 The golden rule for folders: short, no spaces

ESP-IDF’s build tools are fussy about folder names. A space in a path can make a tool think one name is two separate words, and very long paths can go over Windows’ length limit once the build creates its own deeply nested folders inside. Keep both the ESP-IDF install folder and your project folders short, and use underscores instead of spaces.

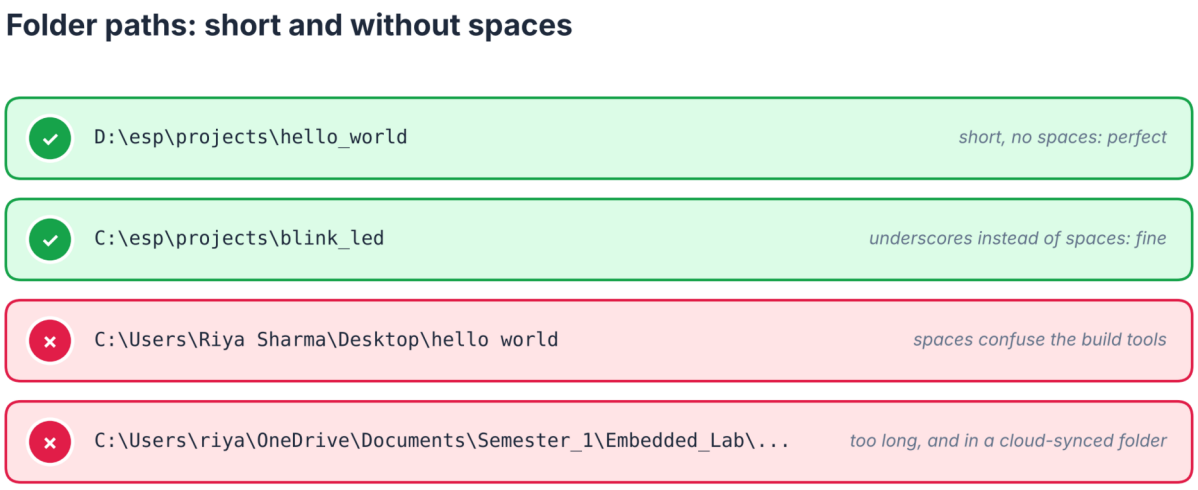


Figure 8. *Good and bad places for ESP-IDF projects.*

> **COMMON MISTAKE**
>
> Do not put projects on the Desktop or inside a OneDrive folder. Your Windows user name often contains a space, and cloud-sync programs can lock files while the build is trying to write them.

> NEW WORDS IN THIS SECTION
>
> **Path:** the full address of a file or folder, such as `D:\esp\projects\hello_world`. Each `\` separates one folder from the next.
>
> **Offline installer:** an installer that contains every needed file, so it doesn't download anything during installation.

> NOTE
>
> **Using a different chip?** ESP-IDF v5.5.5 builds for all the chips in the table in Chapter 6.3. For a chip newer than that, Chapter 6.6 shows how to find the matching ESP-IDF and Arduino core versions yourself.

You will check that the installation worked in Step 1 of the lab.

CHAPTER 4

Terminal survival kit

4.1 What is a terminal?

A **terminal** is a window where you control the computer by typing **commands** instead of clicking. The text at the start of each line, such as `PS D:\esp>`, is the **prompt**: the computer's way of saying "I'm in this folder, ready for your next command". You type a command after it and press **Enter**.

A command has parts. Let's take the one you will use at the end of this lab apart:

EXAMPLE COMMAND

```
idf.py -p COM19 flash monitor
```

Part	Name	Meaning
`idf.py`	the program	Which program to run.
`-p COM19`	an option with a value	Options usually start with `-` or `--`. Here `-p` means "port" and `COM19` is its value.
`flash monitor`	actions	What we want idf.py to do, in order: first flash, then monitor.

Together, the parts after the program name are called **arguments**.

4.2 The commands you'll use in this lab

Type this	What it does
`cd D:\esp\projects`	*Change directory*: move into a folder. (Directory is another word for folder.)
`mkdir D:\esp\projects`	*Make directory*: create a new folder.
`dir`	List the files and folders in the current folder.
`type main\main.cpp`	Print a text file's contents in the terminal.
`notepad main\idf_component.yml`	Open a file in Notepad to edit it.
`Select-String`	Search inside a file for some text (PowerShell only).
Up arrow key	Bring back your previous command, so you don't retype it.
Tab key	Auto-complete a file or folder name you have started typing.
Ctrl + C	Stop the command that is currently running.
Ctrl +]	Exit the ESP-IDF serial monitor.

4.3 PATH: how Windows finds a program

When you type `idf.py`, how does Windows know where that program is? It keeps a list of folders called **PATH**. Every time you type a command, Windows checks each PATH folder in order until it finds a program with that name. If it checks them all and finds nothing, you get an error like "**'idf.py' is not recognized**".

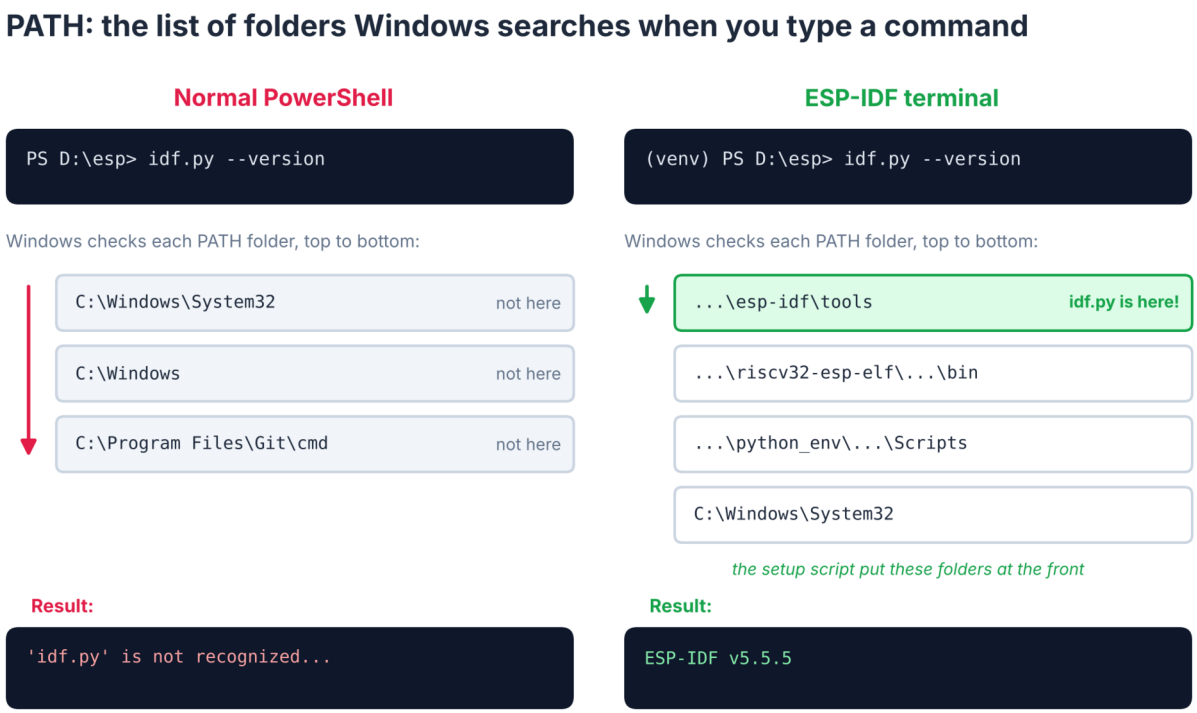


Figure 9. *A normal terminal does not have ESP-IDF's folders in PATH. The ESP-IDF terminal runs a setup script that adds them first.*

This is why we always use the **ESP-IDF terminal** from the Start menu. When it opens, it runs a small *setup script* that adds ESP-IDF's folders (idf.py, the compiler, esptool and more) to the front of PATH. The change only lasts for that window. A normal PowerShell window that you open separately knows nothing about ESP-IDF.

> **THINK OF IT LIKE THIS**
>
> PATH is like your phone's contact list. If you say "call Riya" and Riya is not in your contacts, the phone can't do it, even though Riya exists. The setup script adds ESP-IDF's tools to the contact list for that window.

4.4 Python and the (venv) sign

`idf.py` is written in **Python**, a popular programming language. ESP-IDF keeps its own private copy of Python and its add-on packages in a **virtual environment** ("venv" for short), so that it can never clash with any other Python on your computer. When that private copy is switched on, many ESP-IDF terminals show `(venv)` at the start of the prompt:

AN ESP-IDF TERMINAL PROMPT

```
(venv) PS D:\esp\projects\hello_world>
```

COMMON MISTAKE

Running `idf.py` in an ordinary PowerShell window is the most common beginner mistake. Before any `idf.py` command, glance at the prompt. The most reliable test is `idf.py --version`: if it prints a version, you are in the right terminal.

4.5 idf.py, the team leader

You will type `idf.py` for almost everything in this lab. It does very little work itself. Instead it calls the right specialist tool for each job, like a team leader handing out tasks.



Figure 10. *idf.py calls six specialist tools. Under each tool is the idf.py command that uses it.*

NEW WORDS IN THIS SECTION

Terminal: a text window where you type commands.

Command / argument: an instruction you type; the extra words after the program name are its arguments.

Prompt: the text before your cursor showing that the terminal is ready.

PATH: the list of folders Windows searches to find a program you type.

Python: a programming language; idf.py and esptool are written in it.

Virtual environment (venv): a private, separate copy of Python and its packages used only by ESP-IDF.

CMake / Ninja: build tools: CMake plans what must be compiled, Ninja carries out the plan quickly.

CHAPTER 5

The lab: Hello world in 8 steps

Here is the whole journey. Steps 1 to 5 are done once for each new project. After that, changing your program is just edit, build and flash.

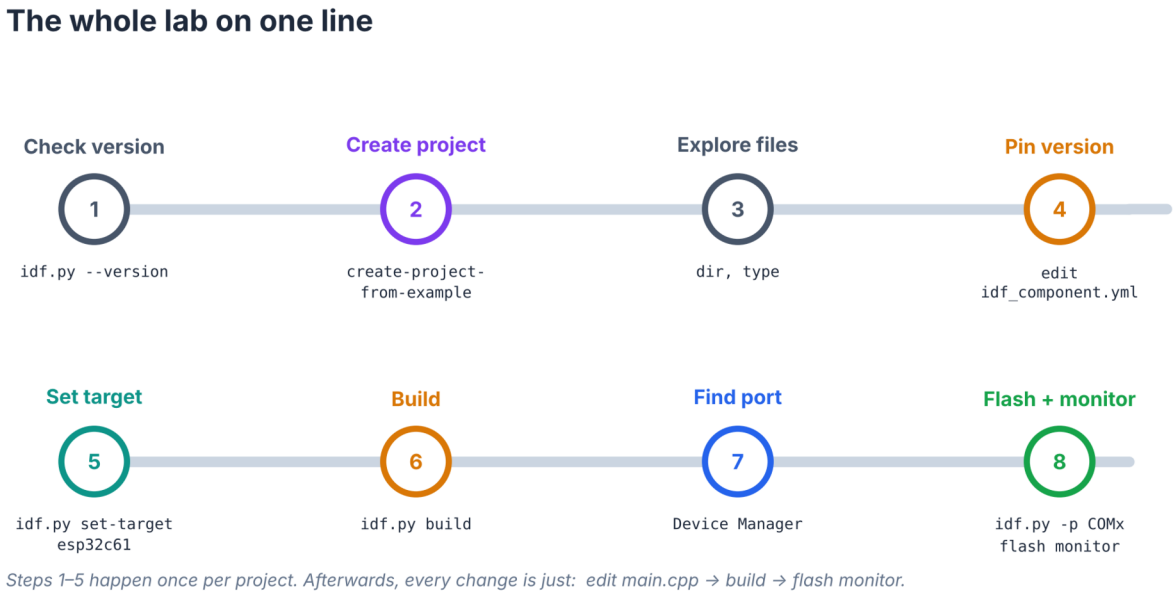


Figure 11. *The eight steps of this lab, with the command or tool used in each.*

Each step follows the same pattern: **Goal** (what we want), **Why** (the reason), **Do it** (the exact commands), **What you should see**, and then an explanation of what just happened. Don’t skip the explanations: they are what will let you fix problems on your own.

STEP 1

Open the ESP-IDF terminal and check the version

Goal: prove that ESP-IDF v5.5.5 is installed and that you are in the right kind of terminal.

WHY ARE WE DOING THIS?

Two things must be true before anything else can work: Windows must be able to *find* ESP-IDF's tools (that is the PATH idea from Chapter 4), and the installed ESP-IDF must be the *version* that fits our Arduino core (Chapter 3). One command checks both.

Do it

1. Press the **Windows** key and type `ESP-IDF`.
2. Open the entry called something like **ESP-IDF 5.5 PowerShell** or **ESP-IDF 5.5 CMD**. Either works. (This manual shows PowerShell.)
3. Wait for the setup script to finish, then type:

TYPE THIS

```
idf.py --version
```

YOU SHOULD SEE (similar to this)

```
ESP-IDF v5.5.5
```

`--version` is an option that asks a program to print its version number and stop.

COMMON MISTAKE

"'idf.py' is not recognized" means you are in an ordinary terminal, not the ESP-IDF one, so PATH doesn't include ESP-IDF's folders. Close it and open the ESP-IDF entry from the Start menu.

A different version (such as v6.0 or v5.4) means the wrong ESP-IDF is installed or you opened the shortcut for another version. Install v5.5.5 (Chapter 3); different versions can live side by side, each with its own Start-menu entry.

STEP 2

Create the project

Goal: make a new project folder that already has the Arduino core set up as a component.

> **WHY ARE WE DOING THIS?**
>
> In ESP-IDF, a **project** is simply a folder that holds your code plus a few small text files telling ESP-IDF how to build it. You will have one project folder per program. Instead of writing those files by hand, we start from a ready-made example that already includes the Arduino core.

Do it

First make a folder for all your projects and move into it. Use a short path with no spaces. These examples use drive D:, but C: is fine too.

TYPE THIS

```
mkdir D:\esp\projects
cd D:\esp\projects
```

Now create the project. You need internet for this:

TYPE THIS

```
idf.py create-project-from-example "espressif/arduino-esp32^3.3.12:hello_world"
```

Every part of that long command means something:

Part	Meaning
`create-project-from-example`	Download an example and turn it into a new project folder.
`espressif/arduino-esp32`	The component's name in the registry: who published it / what it is called.
`^3.3.12`	Which version to take the example from: 3.3.12 or a newer 3.x version (more on `^` below).
`:hello_world`	Which example inside that component we want.

Where does it download from?

Espressif runs an online catalogue called the **ESP Component Registry** (components.espressif.com). It is a bit like the Arduino Library Manager, but for ESP-IDF components. A tool built into ESP-IDF, the **Component Manager**, downloads things from it for you.

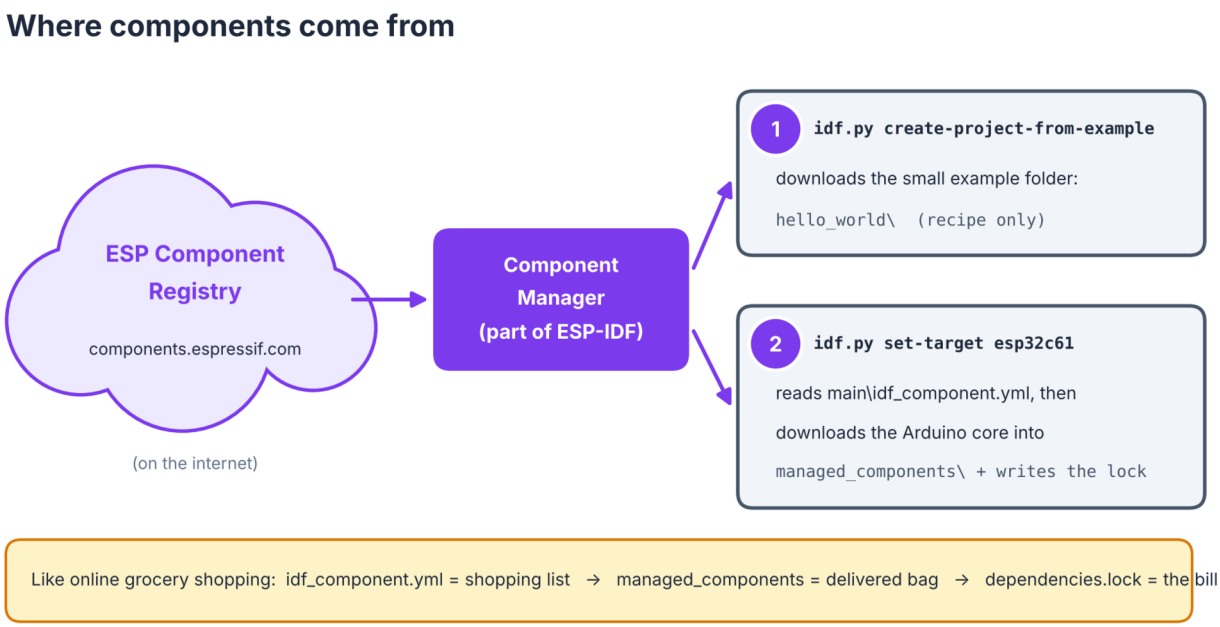


Figure 12. *The Component Manager fetches things from the online registry twice in this lab: the example now, and the Arduino core in Step 5.*

Notice that right now you only get the small example folder: the *recipe*. The Arduino core itself (the ingredients) is downloaded later, in Step 5.

> **NOTE**
> This example is not tied to any chip. The same command creates the starting project for an ESP32-C2, C3, S3 or any other ESP chip; the chip is chosen later, in Step 5.

Reading version numbers

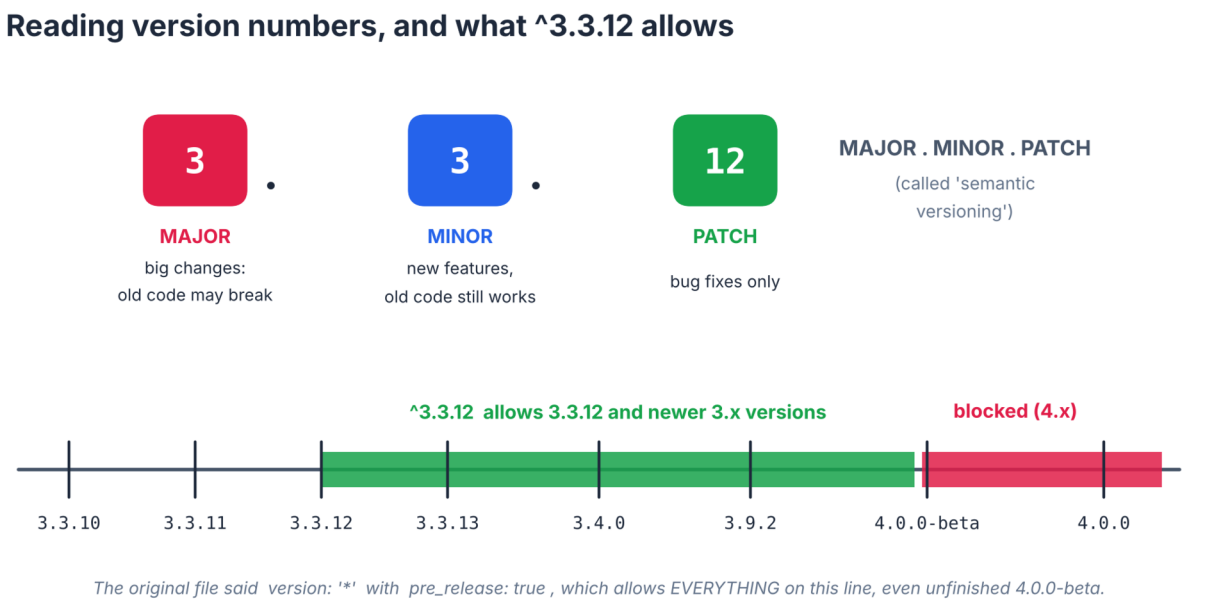


Figure 13. *Version numbers have three parts. The ^ rule accepts bug fixes and new features, but not a new major version that might break your code.*

Most software uses **semantic versioning**: MAJOR.MINOR.PATCH. A change in the last number means only bug fixes. A change in the middle number adds features without breaking old code. A change in the first number may break old code. So `^3.3.12` (read "caret 3.3.12") means "3.3.12 or anything newer, as long as it still starts with 3".

Look at what you got

TYPE THIS

```
cd hello_world
dir
dir main
```

YOU SHOULD SEE (similar to this)

```
    Directory: D:\esp\projects\hello_world
Mode       LastWriteTime        Length Name
d-----     ...                         main
-a----     ...                    407 CMakeLists.txt
-a----     ...                   2653 README.md
-a----     ...                    359 sdkconfig.defaults

    Directory: D:\esp\projects\hello_world\main
-a----     ...                     77 CMakeLists.txt
-a----     ...                     80 idf_component.yml
-a----     ...                    129 main.cpp
```

In the `Mode` column, `d` marks a folder (directory) and `a` marks an ordinary file. `Length` is the file size in bytes. These files are tiny: the whole project is under 4 KB so far.

> NEW WORDS IN THIS SECTION
>
> **Project:** a folder containing your code and the files that tell ESP-IDF how to build it.
>
> **ESP Component Registry:** Espressif's online catalogue of components and examples.
>
> **Component Manager:** the part of ESP-IDF that downloads components from the registry.
>
> **Semantic versioning:** the MAJOR.MINOR.PATCH way of numbering versions.
>
> **Byte / KB / MB:** a byte is 8 bits (one small number from 0 to 255). 1 KB = 1024 bytes; 1 MB = 1024 KB.

STEP 3

Explore the project files

Goal: understand what every file in the project is for, before we change or build anything.

WHY ARE WE DOING THIS?

When something goes wrong later, knowing which file controls what is the difference between guessing and fixing. This is also the step where several important ideas (settings, FreeRTOS, header files) first appear.

Anatomy of the project folder (after the first build)

hello_world	the project = one folder	
├─ CMakeLists.txt	"I am an ESP-IDF project called main"	rarely edit
├─ sdkconfig.defaults	your starting settings	you edit
├─ sdkconfig	ALL settings (created on first configure)	made for you
├─ dependencies.lock	exact versions that were downloaded	made for you: keep it
├─ README.md	notes for humans; the build ignores it	
├─ **main**	YOUR component	
│ ├─ CMakeLists.txt	registers main.cpp with the build	rarely edit
│ ├─ idf_component.yml	dependency list: needs arduino-esp32	you edit
│ └─ main.cpp	your sketch lives here	you edit
├─ **managed_components**	downloaded by the Component Manager	never edit
│ └─ espressif__arduino-esp32\	the Arduino core	never edit
└─ **build**	compiled output; recreated each build	safe to delete

Figure 14. *The project folder as it will look after your first build. The tags tell you which files you edit and which you must leave alone.*

There are two levels. The **project level** is the `hello_world` folder itself. The **component level** is the `main` folder inside it. ESP-IDF treats `main` as a component too: the one that holds *your* code. So your project is your `main` component plus the Arduino component, which is downloaded later.

File	What it is
`CMakeLists.txt` (project)	Instructions for **CMake**, the build system. Mostly says "this is an ESP-IDF project, and its name is main". You will rarely edit it.
`README.md`	A description for humans, in **Markdown** (a simple text format using symbols like `#` for headings). The build ignores it.
`sdkconfig.defaults`	Starting values for settings. Explained just below.
`main\CMakeLists.txt`	Registers the `main` component: tells the build "these are my source files, compile them".
`main\idf_component.yml`	The component's **dependency** list: what it needs that it doesn't contain itself. Explained in Step 4.
`main\main.cpp`	Your program. This is where the sketch code goes.

> **TRY IT**
> Peek inside the two CMake files with `type CMakeLists.txt` and `type main\CMakeLists.txt`. They are only a few lines each. Can you spot where `main.cpp` is mentioned?

Inside sdkconfig.defaults

TYPE THIS

```
type sdkconfig.defaults
```

YOU SHOULD SEE (similar to this)

```
#
# Arduino ESP32
#
CONFIG_AUTOSTART_ARDUINO=y
# end of Arduino ESP32

#
# FREERTOS
#
CONFIG_FREERTOS_HZ=1000
...
# IDF v6 requirements
CONFIG_LIBC_NEWLIB=y
CONFIG_COMPILER_DISABLE_DEFAULT_ERRORS=y
```

ESP-IDF has hundreds of settings, such as Wi-Fi buffer sizes or clock speed. Each one is written as `CONFIG_NAME=value`, where `y` means *yes* (switched on). Lines starting with `#` are **comments**: notes for humans that ESP-IDF ignores.

Setting	What it does, in plain words
`CONFIG_AUTOSTART_ARDUINO=y`	Makes the Arduino component run your `setup()` once and then `loop()` forever, just like a normal Arduino. Without it, ESP-IDF would look for its own starting function instead.
`CONFIG_FREERTOS_HZ=1000`	Sets the FreeRTOS "heartbeat" to 1000 ticks per second. Arduino's `delay()` and `millis()` count in milliseconds and need this. See below.
`CONFIG_LIBC_NEWLIB=y`	Picks which C standard library (basic functions like `printf`) to use. It exists for ESP-IDF v6; on our v5.5.5 it changes nothing, and you may see a harmless warning about it.
`CONFIG_COMPILER_DISABLE_DEFAULT_ERRORS=y`	ESP-IDF normally treats some compiler *warnings* as *errors* to be strict. Many Arduino libraries trigger those warnings, so this keeps them as warnings and lets the build finish.

FreeRTOS: one processor, many jobs

Underneath ESP-IDF runs **FreeRTOS**, a small **operating system** for microcontrollers. An operating system is software that manages the chip and lets several jobs, called **tasks**, share the processor. The ESP32-C61 has a single processor core, so only one task can truly run at any instant. FreeRTOS switches between tasks so quickly that it looks like they all run at the same time.

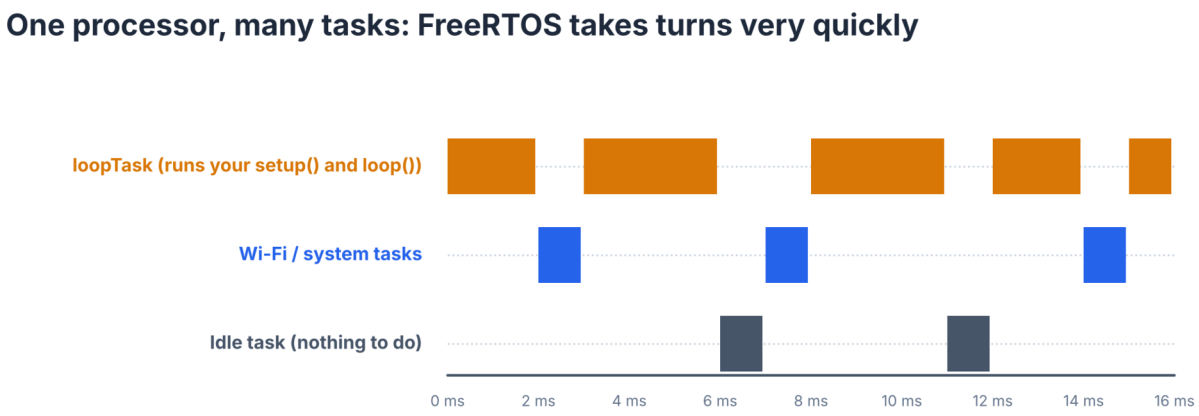


Figure 15. *Your setup() and loop() run inside a task called loopTask. FreeRTOS gives each task short turns on the processor.*

> **THINK OF IT LIKE THIS**
>
> A teacher with a class of students who all have questions can answer only one at a time. If she answers each one for a few seconds and moves on very quickly, everyone feels looked after. FreeRTOS is that teacher, and the tasks are the students.

FreeRTOS keeps time with a regular **tick**, like a heartbeat. At 1000 Hz (hertz, meaning *times per second*) there is one tick every millisecond, so a 1-millisecond `delay(1)` can work as promised.

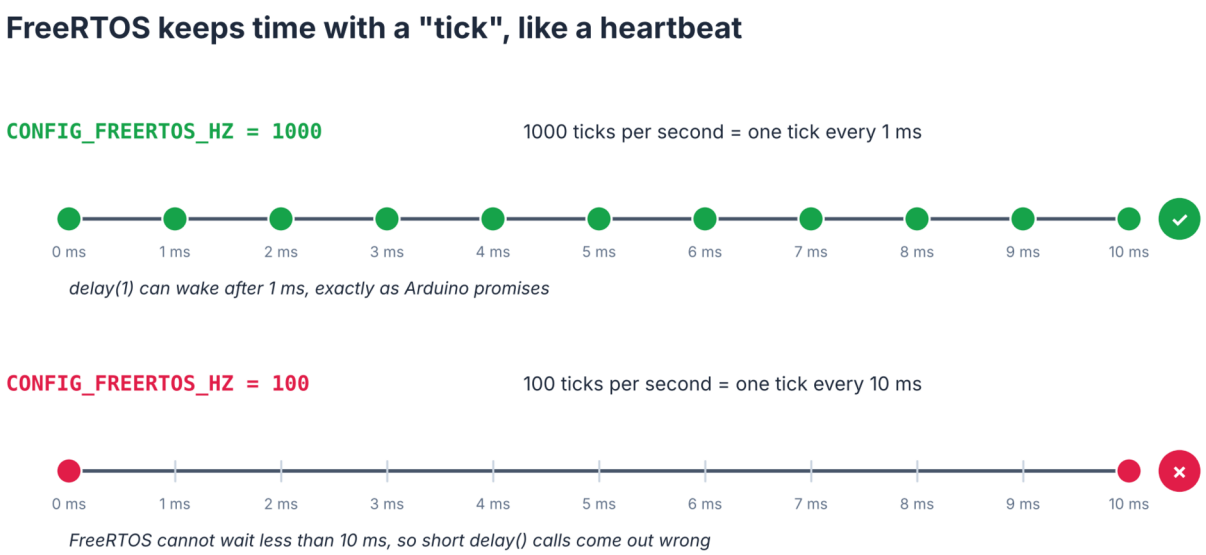


Figure 16. *With 1000 ticks per second FreeRTOS can measure single milliseconds; with the ESP-IDF default of 100 it can't.*

Inside main.cpp

TYPE THIS

```
type main\main.cpp
```

YOU SHOULD SEE (similar to this)

```
#include "Arduino.h"

void setup() {
  Serial.begin(115200);
}

void loop() {
  Serial.println("Hello world!");
  delay(1000);
}
```

- `#include "Arduino.h"` tells the compiler to pull in another file first. `Arduino.h` is a **header file**: it declares which functions exist (`Serial`, `delay`, `digitalWrite`, ...) so the compiler recognises those names. **The Arduino IDE adds this line to every sketch silently. ESP-IDF does not**, so every sketch you bring over will need it at the top.
- `Serial.begin(115200)` starts the serial connection: the text channel between the board and your computer over the USB cable. `115200` is the **baud rate**, how many bits per second are sent. Both ends must agree on this number (Step 8 shows what happens if they don't).
- `loop()` prints "Hello world!", waits 1000 milliseconds (one second), and repeats forever.

NEW WORDS IN THIS SECTION

Build system: a program that works out which files to compile, in what order, with which settings, and then runs the compiler.

Dependency: something your code needs that it doesn't contain itself.

Operating system: software that manages the hardware and shares the processor between tasks.

Task: one job that FreeRTOS schedules; your setup()/loop() run in one called loopTask.

Tick: FreeRTOS's regular timing heartbeat.

Header file: a file (usually .h) that declares functions so other code can use them.

Baud rate: the speed of a serial connection, in bits per second.

STEP 4

Pin the Arduino core version

Goal: make the project always use Arduino core 3.3.x, starting from 3.3.12.

What the file says now

TYPE THIS

```
type main\idf_component.yml
```

YOU SHOULD SEE (similar to this)

```
dependencies:
  espressif/arduino-esp32:
    pre_release: true
    version: '*'
```

- `dependencies:` starts the list of things this component needs.
- `espressif/arduino-esp32:` names the dependency.
- `version: '*'` means **any version at all**.
- `pre_release: true` also allows **pre-release** versions: unfinished test versions (often labelled alpha, beta or rc) published so developers can try them early.

> **WHY ARE WE DOING THIS?**
> Together these lines say "give me any Arduino core, even an unfinished one". Two problems: a future test version of Arduino core 4.0 might be built for ESP-IDF v6 and fail on your v5.5.5; and two students building the same project a month apart could silently get different versions. For a class we want **reproducible** builds: the same project should always produce the same result.

> **NOTE**
> The `^3.3.12` you typed in Step 2 only chose *which copy of the example* to download. The project's own dependency list inside the example is a separate thing, and that is what we are fixing now.

Do it

TYPE THIS

```
notepad main\idf_component.yml
```

Delete everything in the file and replace it with exactly these three lines. Then save (Ctrl + S) and close Notepad.

NEW CONTENTS OF idf_component.yml

```
dependencies:
  espressif/arduino-esp32:
    version: '^3.3.12'
```

YAML and indentation

The file uses **YAML**, a simple text format for settings (`.yml` is its file ending). YAML uses **indentation**, the spaces at the start of a line, to show what belongs to what. Here `version` is indented further than `espressif/arduino-esp32`, which shows it belongs to it.

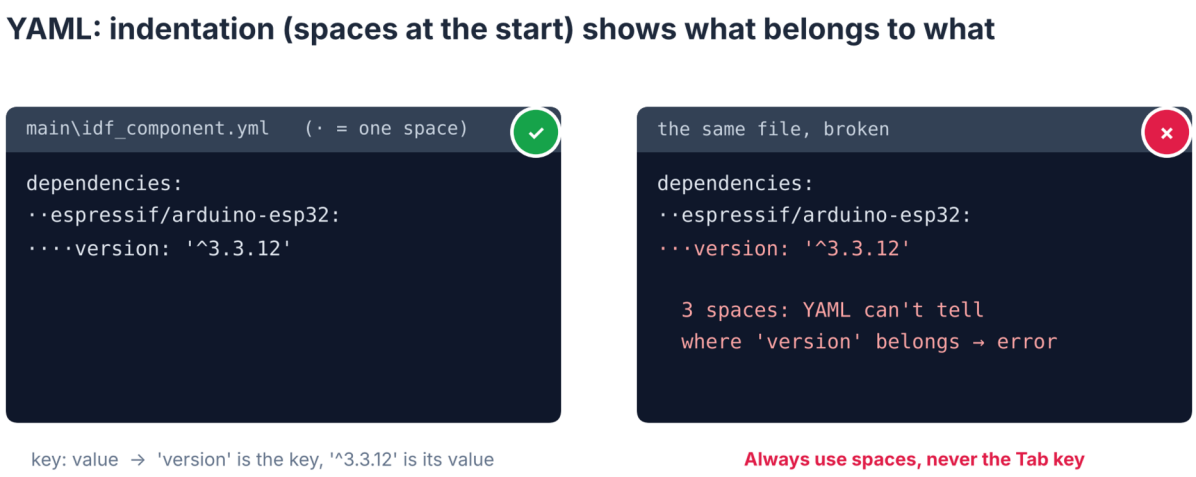


Figure 17. *Two spaces before espressif, four before version. Getting the spaces wrong breaks the file.*

COMMON MISTAKE

Never press the **Tab** key to indent in YAML. Tabs are not allowed. Use the space bar: exactly 2 spaces before `espressif` and 4 before `version`.

NEW WORDS IN THIS SECTION

Pre-release: an unfinished test version, published early for developers to try.

Reproducible build: the same project always produces the same result, on any computer, on any day.

YAML: a simple text format for settings that uses indentation to show structure.

Indentation: spaces at the start of a line.

STEP 5

Set the target chip

Goal: tell ESP-IDF we are building for the ESP32-C61, and let it download the Arduino core.

> **WHY ARE WE DOING THIS?**
>
> ESP-IDF supports many chips: ESP32, S3, C3, C6, C61 and more. They have different processors and different hardware inside. The chip you are building for is called the **target**, and ESP-IDF needs to know it before it can choose the right compiler and settings.

Do it

Make sure you are still inside the `hello_world` folder, then run:

TYPE THIS

```
idf.py set-target esp32c61
```

This takes a few minutes and prints a lot of text. You don't need to read it all. It has worked when the last lines look like this:

YOU SHOULD SEE (similar to this)

```
-- Configuring done (87.2s)
-- Generating done (2.1s)
-- Build files have been written to: D:/esp/projects/hello_world/build
```

> **NOTE**
>
> **Using a different chip?** This is the only command that changes. Replace `esp32c61` with your chip's target name, for example `esp32c2`, `esp32c3` or `esp32s3`. Chapter 6.3 lists all of them, and `idf.py --list-targets` prints the ones your ESP-IDF supports.

What just happened: three jobs in one command

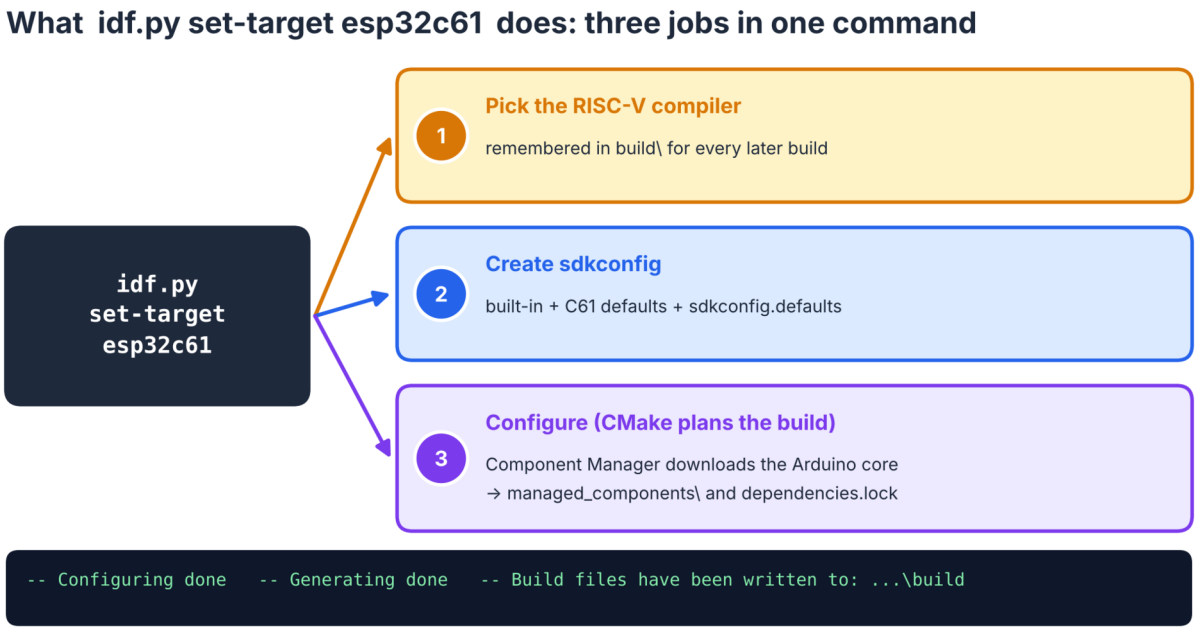


Figure 18. *set-target picks the compiler, creates the settings file and runs the configure stage, which downloads the Arduino core.*

1. It picks the compiler. The ESP32-C61's processor uses an instruction design called **RISC-V** (say "risk-five"). The original ESP32 and the S3 use a different design called **Xtensa**. Machine code made for one design is meaningless to the other, so each family needs its own compiler.

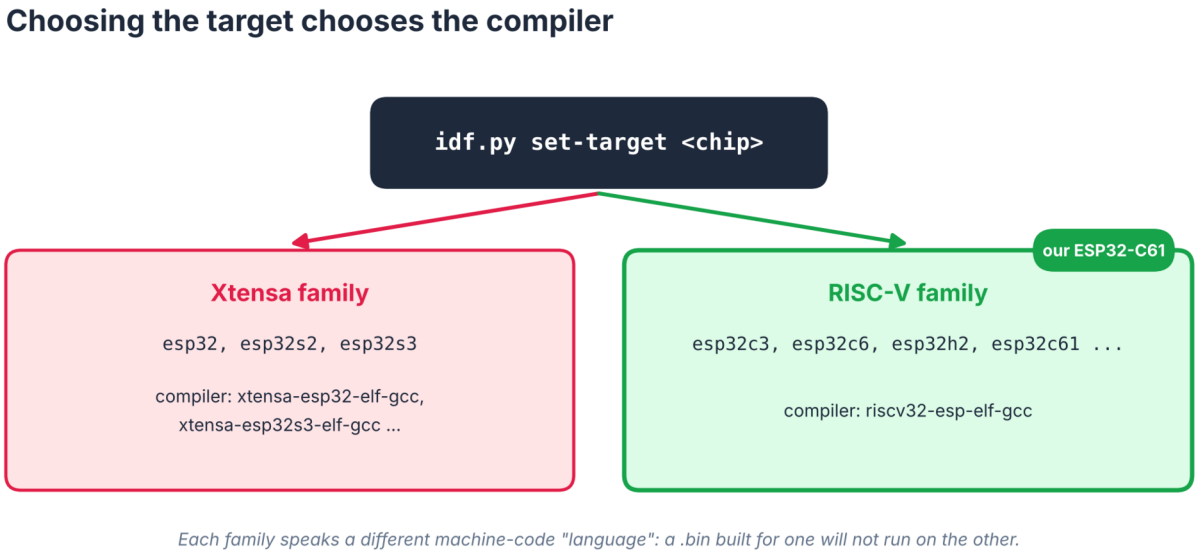


Figure 19. *The target decides which compiler is used. Our ESP32-C61 belongs to the RISC-V family.*

2. It creates the full settings file, `sdkconfig`. ESP-IDF combines its own built-in defaults, the defaults for the C61 chip, and the project's `sdkconfig.defaults`, and writes out every single setting.

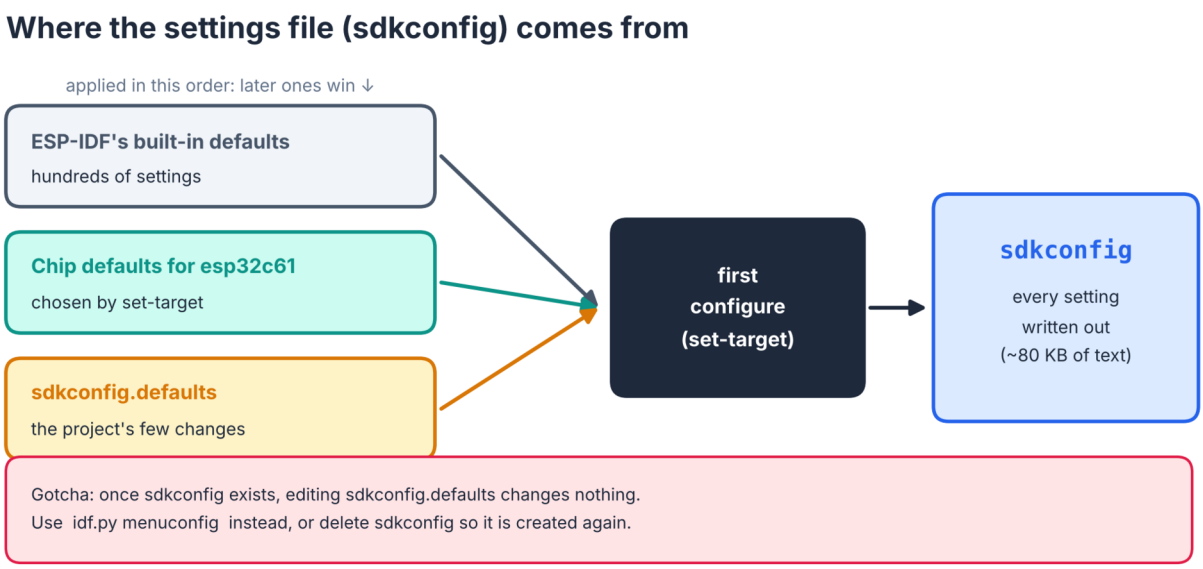


Figure 20. *How sdkconfig is built, and the one gotcha that surprises almost everyone.*

3. It runs CMake's configure stage. This is the planning stage that comes before compiling. During it, the Component Manager reads `main\idf_component.yml` and downloads the Arduino core (and the components the Arduino core itself depends on). That download is most of the 87 seconds, and it happens only once per project.

Look at the new items

```
TYPE THIS
dir
```

New item	What it is	Rule
`managed_components\`	The downloaded Arduino core and its own dependencies. "Managed" means the Component Manager controls it.	**Never edit.** Your changes can be overwritten at any time.
`dependencies.lock`	A record of the *exact* version picked within your `^3.3.12` rule.	**Keep it with the project.** Anyone who builds the project gets exactly the same versions.
`sdkconfig`	The full settings file (about 80 KB of text).	Change it with `idf.py menuconfig`, not by hand.
`build\`	A working folder for compiled files.	**Safe to delete.** It is recreated automatically.

TRY IT

Which Arduino core version did you really get? The core writes its version inside one of its own files. Search that file:

TYPE THIS

```
Select-String -Path managed_components\espressif__arduino-
esp32\cores\esp32\esp_arduino_version.h -Pattern "define ESP_ARDUINO_VERSION_(MAJOR|
MINOR|PATCH)"
```

This is one long command: type it all on *one* line, even though it wraps here. You should see three lines ending in `3`, `3` and `12`: version 3.3.12. Each line is a `#define`, which gives a fixed name to a value. Notice the folder name has a *double* underscore: the Component Manager names folders as publisher__name.

NEW WORDS IN THIS SECTION

Target: the chip you are building for.

RISC-V / Xtensa: two different processor designs (instruction sets). The C61 is RISC-V.

Configure stage: CMake's planning step that runs before any compiling.

Lock file: a file recording the exact versions used, so builds can be repeated.

STEP 6

Build

Goal: compile ESP-IDF, the Arduino core and your code into binary files for the ESP32-C61.

> **COMMON MISTAKE**
>
> Before building, look at your prompt. If you closed the ESP-IDF terminal and opened a *normal* PowerShell, commands like `dir` and `Select-String` still work, but `idf.py build` will fail with "not recognized". Open the ESP-IDF terminal and `cd D:\esp\projects\hello_world` again.

Do it

TYPE THIS

```
idf.py build
```

- **The first build is slow**: 5 to 15 minutes, because all of ESP-IDF and the whole Arduino core are compiled for the C61. Thousands of files.
- **Lots of yellow warning lines are normal.** A **warning** is the compiler saying "this looks suspicious"; an **error** is it refusing to continue. Only errors stop the build.

YOU SHOULD SEE (similar to this)

```
Generating binary image from built executable
esptool.py v4.12.0
Creating esp32c61 image...
Successfully created esp32c61 image.
Generated D:/esp/projects/hello_world/build/main.bin
[1576/1576] ...
main.bin binary size 0x3f0a0 bytes. Smallest app partition is 0x100000 bytes. 0xc0f60
bytes (75%) free.

Project build complete. To flash, run:
 idf.py flash
or
 python -m esptool --chip esp32c61 -b 460800 --before default_reset --after hard_reset
   write_flash --flash_mode dio --flash_size 2MB --flash_freq 80m 0x0
build\bootloader\bootloader.bin
   0x8000 build\partition_table\partition-table.bin 0x10000 build\main.bin
```

What happens inside a build

The single word "compiling" from Chapter 1 really hides three stages:

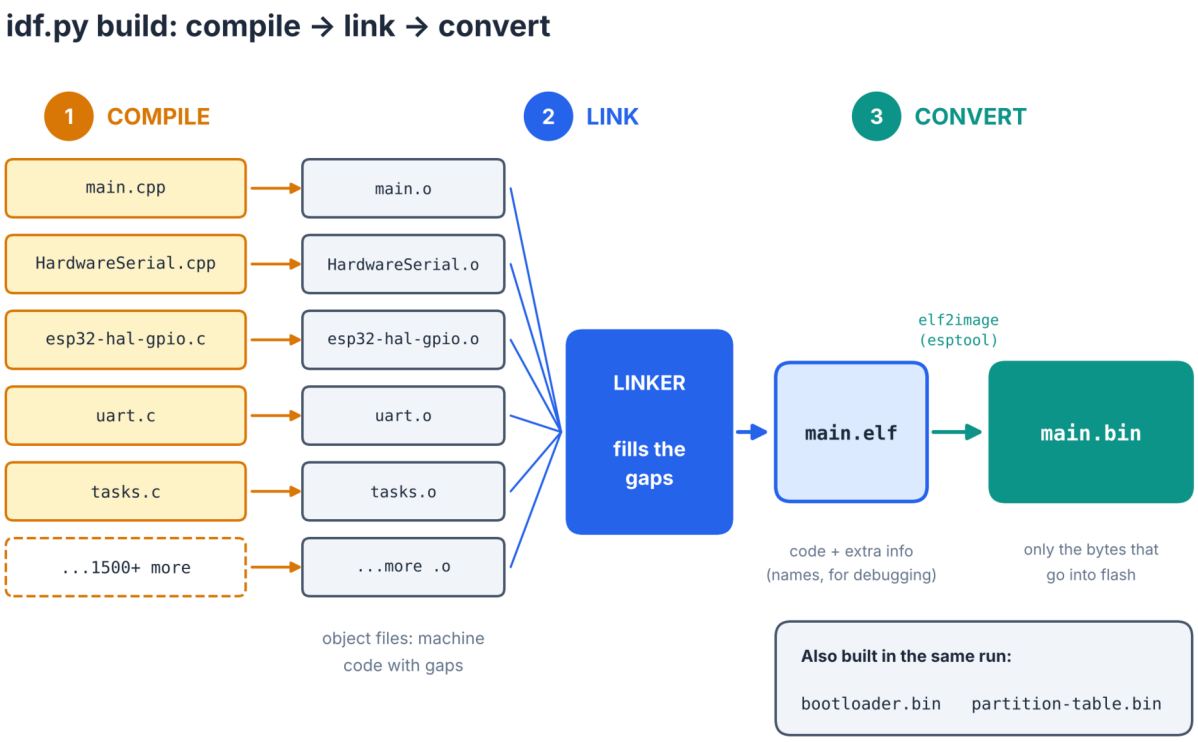


Figure 21. *A build compiles every source file to an object file, links them into main.elf, then converts that into main.bin.*

1. **Compile.** Each `.c` or `.cpp` file is translated separately into an **object file** (`.o`). An object file is machine code for that one file, but with gaps: wherever it uses something defined in another file, the compiler doesn't yet know where that thing will be.
2. **Link.** The **linker** joins all the object files into one program and fills the gaps, connecting every call to where that function actually lives. The result is an **ELF** file (`main.elf`, Executable and Linkable Format): the machine code plus extra information, like function names, that helps when hunting bugs.
3. **Convert.** esptool's `elf2image` command turns `main.elf` into `main.bin`: only the exact bytes that go into flash, with everything else stripped out.

The linker connects calls to where the code really lives

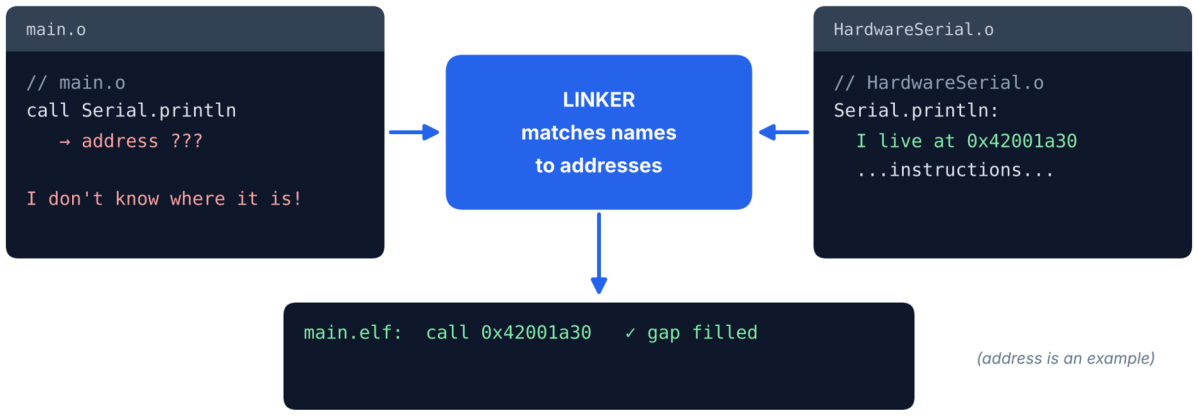


Figure 22. *main.o knows it wants Serial.println but not where it is. The linker finds it in HardwareSerial.o and fills in the address.*

> **FUN FACT**
> Stage 3 should look familiar. Your Arduino IDE error was `esptool elf2image` failing, because the ESP32-C61 bootloader `.elf` file it needed did not exist. This time ESP-IDF built that file itself, from source.

Reading the output

[1576/1576] means the build was 1576 separate jobs, run by **Ninja**. That is why the first build is slow. Next time, if you change only `main.cpp`, only a handful of those jobs run again. This is called an **incremental build**.

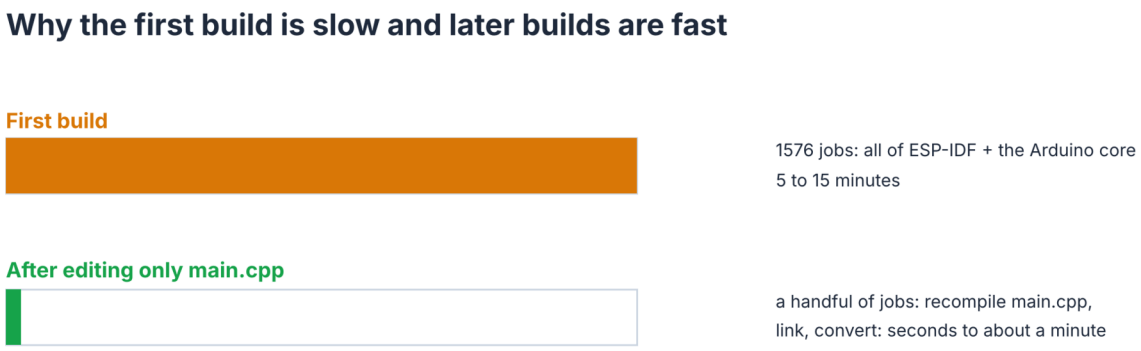


Figure 23. *Ninja only redoes the jobs whose input files changed.*

The size line uses hexadecimal numbers. The `0x` at the start marks a number written in **hexadecimal** (hex for short): base 16, using the digits 0–9 and then the letters a–f for ten to fifteen. Programmers like hex for memory sizes and addresses because each hex digit matches exactly 4 binary bits, so it lines up neatly with how computers count.

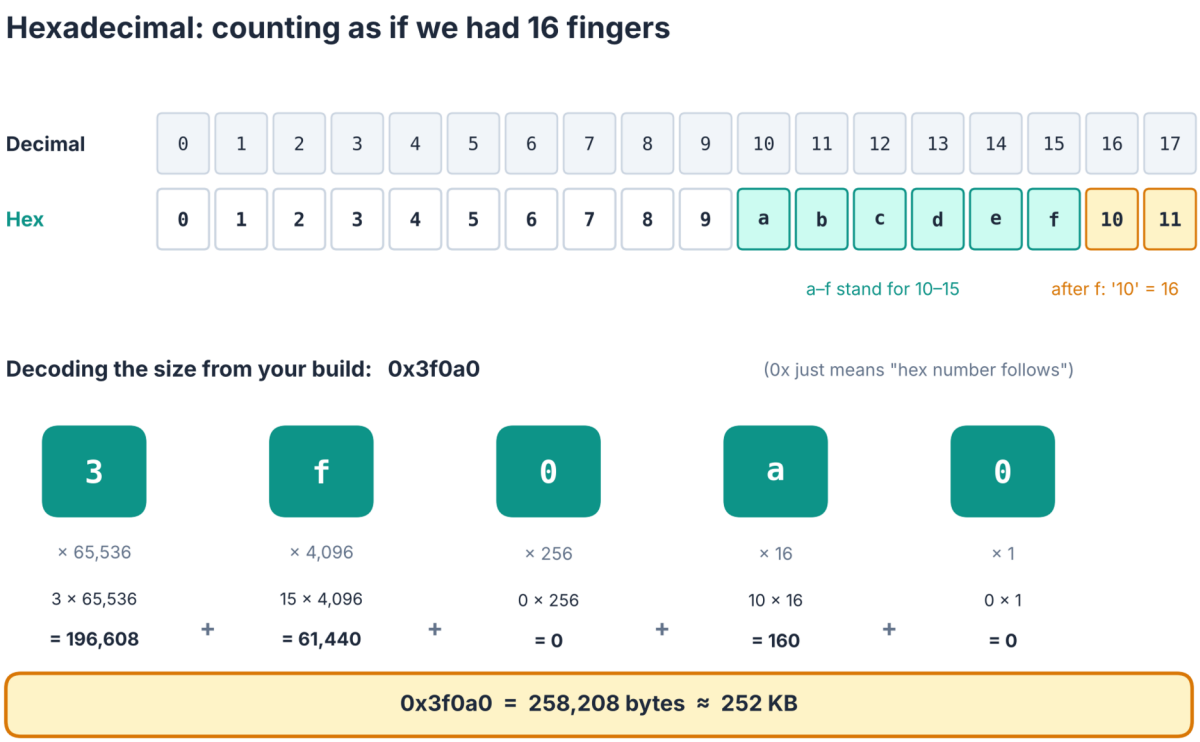


Figure 24. *Counting in hex, and converting the build's 0x3f0a0 into ordinary numbers. Each place is worth 16 times the place to its right.*

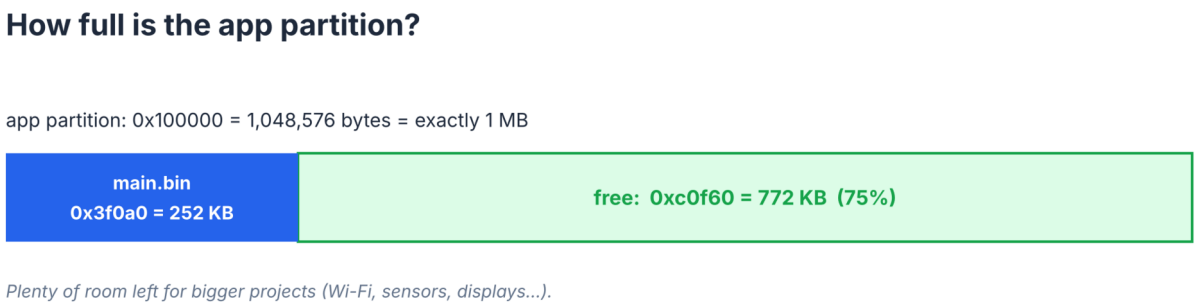


Figure 25. *Your program uses about a quarter of the space reserved for it.*

Hex in the output	In ordinary numbers	Meaning
`0x3f0a0`	258,208 bytes (about 252 KB)	Size of your program, `main.bin`.
`0x100000`	1,048,576 bytes (exactly 1 MB)	Space reserved for a program in flash (the app partition).
`0xc0f60`	790,368 bytes (about 772 KB, 75%)	Space still free in that partition.

> **TRY IT**
>
> Run `idf.py size` to get a breakdown of how much flash and RAM your program uses.

NEW WORDS IN THIS SECTION

Object file (.o): machine code for one source file, with gaps for things defined elsewhere.

Linker: the tool that joins object files and fills in the gaps.

.elf file: the linked program plus extra information for debugging.

Warning / error: a warning is a caution; an error stops the build.

Incremental build: a build that only recompiles what changed.

Hexadecimal (hex): base-16 numbers, written with 0x in front, using digits 0–9 and a–f.

STEP 7

Find your board's port

Goal: find out which COM port Windows has given to your board.

> **WHY ARE WE DOING THIS?**
>
> To flash, esptool needs to know which connection the board is on. When you plug in the board, Windows creates a **port** for it, named like `COM3` or `COM19`. Plug in a different board, or the same board into a different USB socket, and the number can change.

Do it

1. Plug the board in with your USB data cable. If your board has **two** USB ports, use the one labelled **UART** for this lab.
2. Open **Device Manager**: right-click the Start button and choose Device Manager.
3. Expand **Ports (COM & LPT)**. Unplug the board and plug it back in while watching that list. The entry that disappears and comes back is your board.

YOU SHOULD SEE SOMETHING LIKE (your number will differ)

```
Ports (COM & LPT)
    Silicon Labs CP210x USB to UART Bridge (COM19)
```

What is that "CP210x"?

Your computer speaks **USB**, a fast and complicated protocol (set of communication rules). The ESP32-C61's main serial port speaks **UART**, a much simpler one: bits are sent one after another on a single wire. The **USB-to-UART bridge**, here a Silicon Labs CP210x chip, translates between the two. Windows sees it and gives it a COM port.

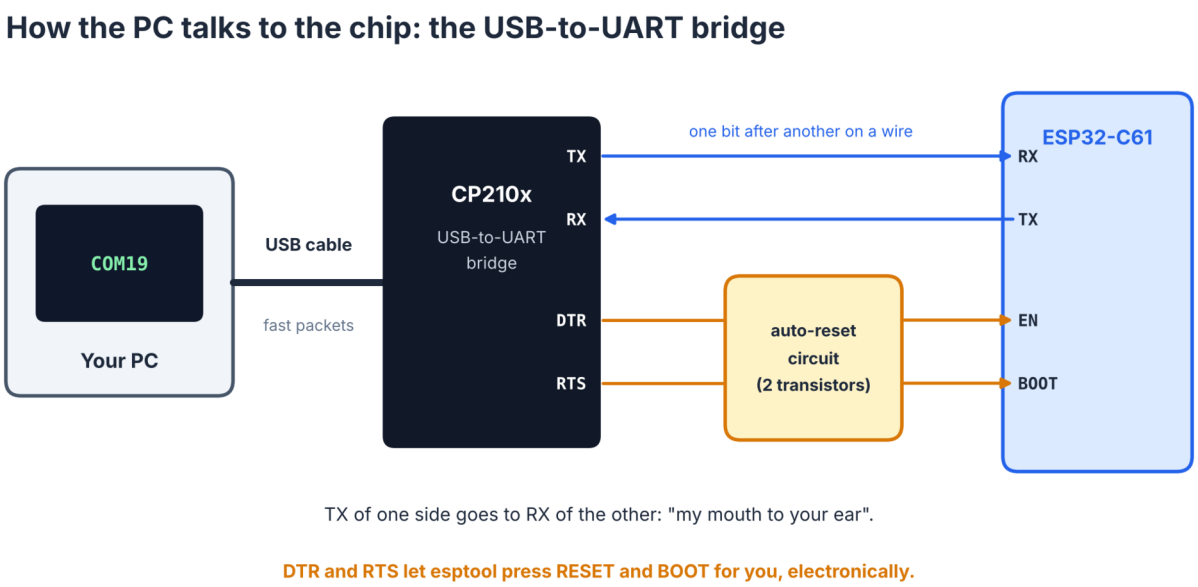


Figure 26. *The CP210x turns USB into UART signals. TX and RX carry data; DTR and RTS are two extra lines used for automatic resetting (Step 8).*

UART uses two data wires: **TX** (transmit) and **RX** (receive). They cross over: one side's TX goes to the other side's RX, like one person's mouth talking to the other person's ear.

Not every board has a bridge chip

Some boards have a **CH340** bridge instead of a CP210x; it does the same job and appears as "USB-SERIAL CH340". Many newer boards have no bridge at all: the USB connector goes straight to the ESP chip, which has USB built in. Those boards appear with a different name and need one extra setting to show `Serial` output on that port (Chapter 6.5).

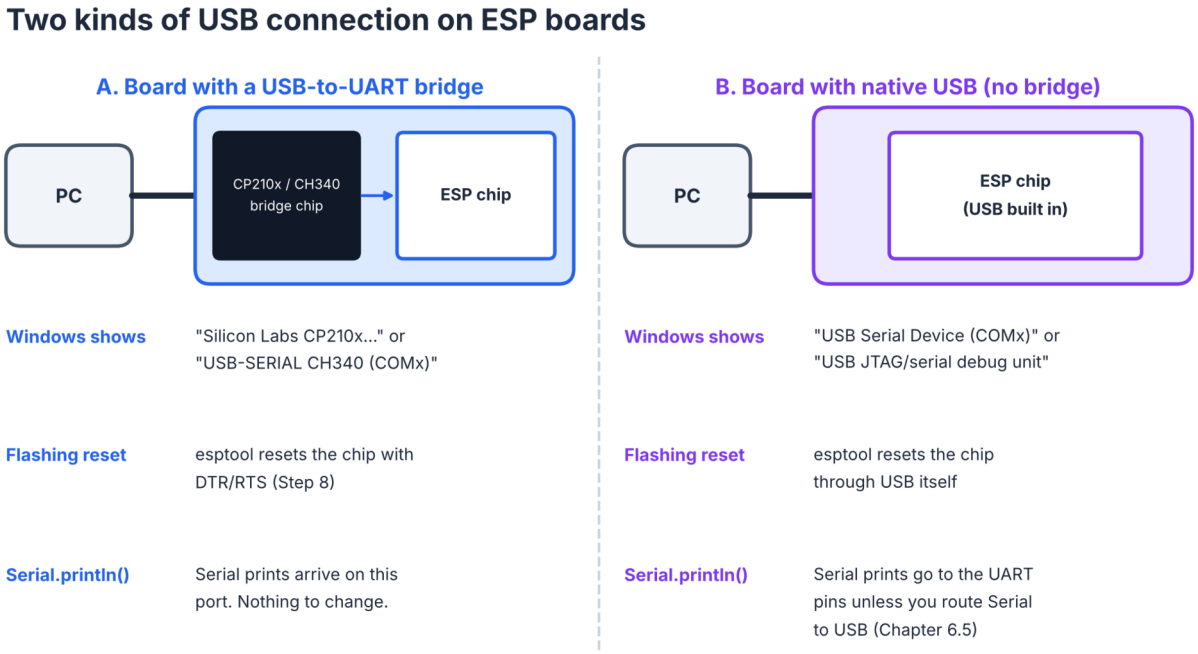


Figure 27. *Bridge-chip boards and native-USB boards look different to Windows and behave slightly differently.*

COMMON MISTAKE

The board shows up under "Other devices" with a yellow warning sign. Windows is missing the CP210x **driver**. Install the "CP210x USB to UART Bridge VCP driver" from the Silicon Labs website, then unplug and replug.

Nothing appears at all. Try another cable: many cables that come with power banks and earphones carry power but no data.

NEW WORDS IN THIS SECTION

Port / COM port: the named connection Windows creates for a serial device, such as COM19.

UART: Universal Asynchronous Receiver-Transmitter: a simple way of sending bits one after another over a wire.

USB-to-UART bridge: a chip that converts between USB (PC side) and UART (chip side).

TX / RX: transmit and receive wires of a UART connection.

Protocol: an agreed set of rules for communicating.

STEP 8

Flash and monitor

Goal: write the program into the board's flash memory and watch it run.

> COMMON MISTAKE
>
> **Close the Arduino IDE's Serial Monitor** (or any other program using the port) first. Only one program can use a COM port at a time. If another one has it open, flashing fails with "Access is denied" or "port busy".

Do it

In the ESP-IDF terminal, inside the project folder (replace `COM19` with your port from Step 7):

TYPE THIS

```
idf.py -p COM19 flash monitor
```

- `-p COM19` sets the port.
- `flash` runs esptool to write the files into flash (and builds first, if anything changed).
- `monitor` then opens a **serial monitor**, like the Arduino IDE's Serial Monitor, showing the text the board sends back.

To exit the monitor, press **Ctrl +]** (the Ctrl key together with the closing square bracket key).

8a. Where do the files go?

Look back at the long esptool command the build suggested. It writes **three** files, each to a specific **address**: a position in flash memory, counted in bytes from the start.

Address	File	What it is
`0x0`	`bootloader.bin`	The second-stage bootloader: the program that starts your program.
`0x8000` (32 KB in)	`partition-table.bin`	The partition table: the map of the flash.
`0x10000` (64 KB in)	`main.bin`	Your program (the app).

The flash is divided into named regions called **partitions**, the way a hard disk is split into C: and D: drives. The **partition table** is the map that says where each partition starts, how big it is and what it is for. Your build uses ESP-IDF's default layout:

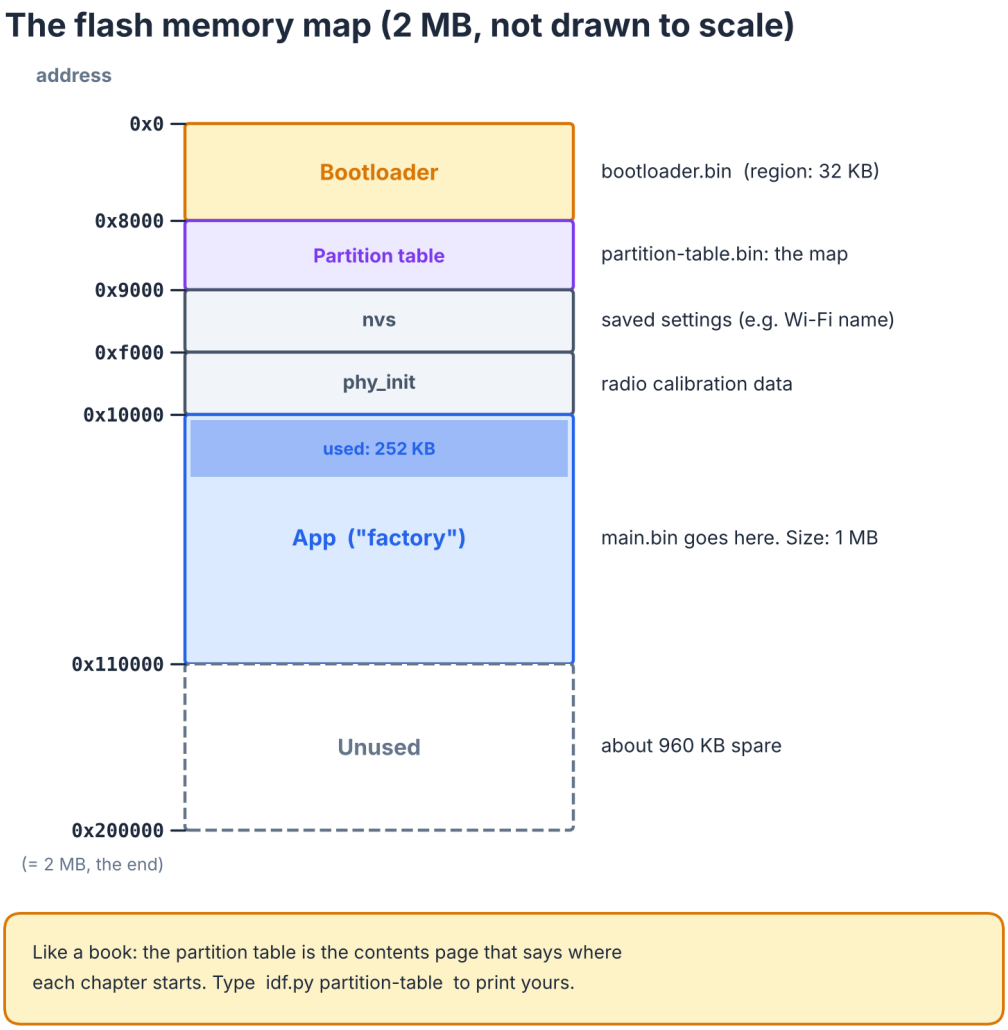


Figure 28. *The default flash layout. nvs stores saved settings (like a Wi-Fi password) and phy_init stores radio calibration data.*

> **NOTE**
>
> The bootloader address depends on the chip: `0x0` on the C61 and most newer chips, `0x1000` on the original ESP32 and the ESP32-S2, `0x2000` on the C5 and P4. You never type it yourself: idf.py always uses the right one for your target.

> **NOTE**
>
> The build assumes a 2 MB flash chip (`--flash_size 2MB`). If your board's flash is bigger, everything still works; the extra space simply stays unused for now.

8b. Getting the chip ready to listen: download mode

Normally, when the chip powers on, a small program built permanently into the chip, the **ROM bootloader**, starts your program straight away. **ROM** means read-only memory: it was written at the factory and can never be changed or erased, which is why a bad flash can never permanently "brick" the chip.

The ROM bootloader has a second mode, **download mode**, in which it does nothing except listen on the serial port for commands like “erase this area” or “write these bytes here”. Which mode it chooses depends on the **BOOT pin** at the exact moment the chip starts:

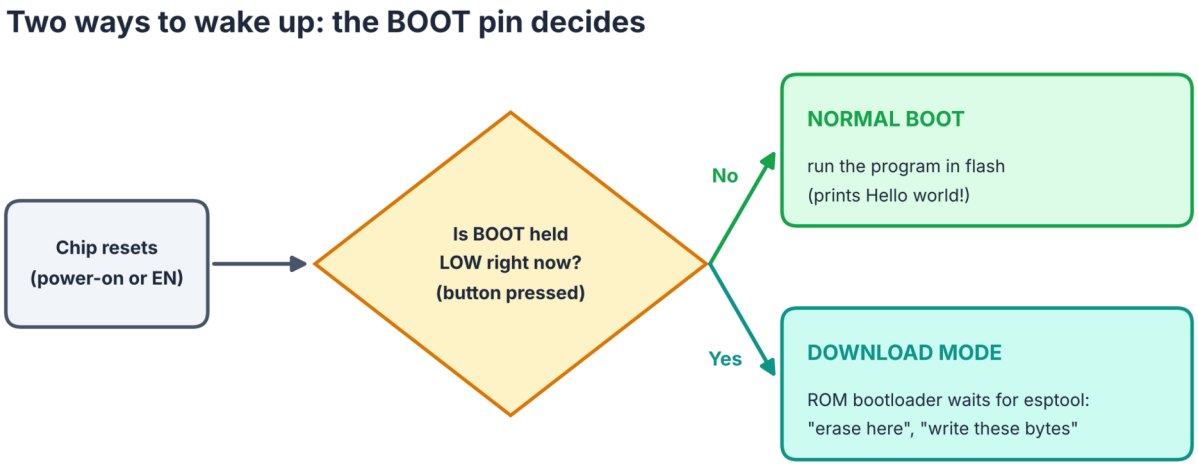


Figure 29. *If BOOT is held low as the chip starts, it waits for esptool; otherwise it runs your program.*

A pin that is checked only at start-up to choose a mode is called a **strapping pin**. Pressing BOOT later, while the program runs, does not change the mode.

8c. Nobody pressed BOOT: how did it get into download mode?

esptool did it for you. Remember the two extra lines on the CP210x, **DTR** and **RTS**? On your board they are wired, through two small transistors (electronic switches), to the chip’s **EN** pin (enable, which works as reset) and its **BOOT** pin. esptool switches DTR and RTS in a quick pattern that is exactly the same as a person holding BOOT, tapping RESET, and letting go of BOOT. That is what `--before default_reset` in the suggested command meant.

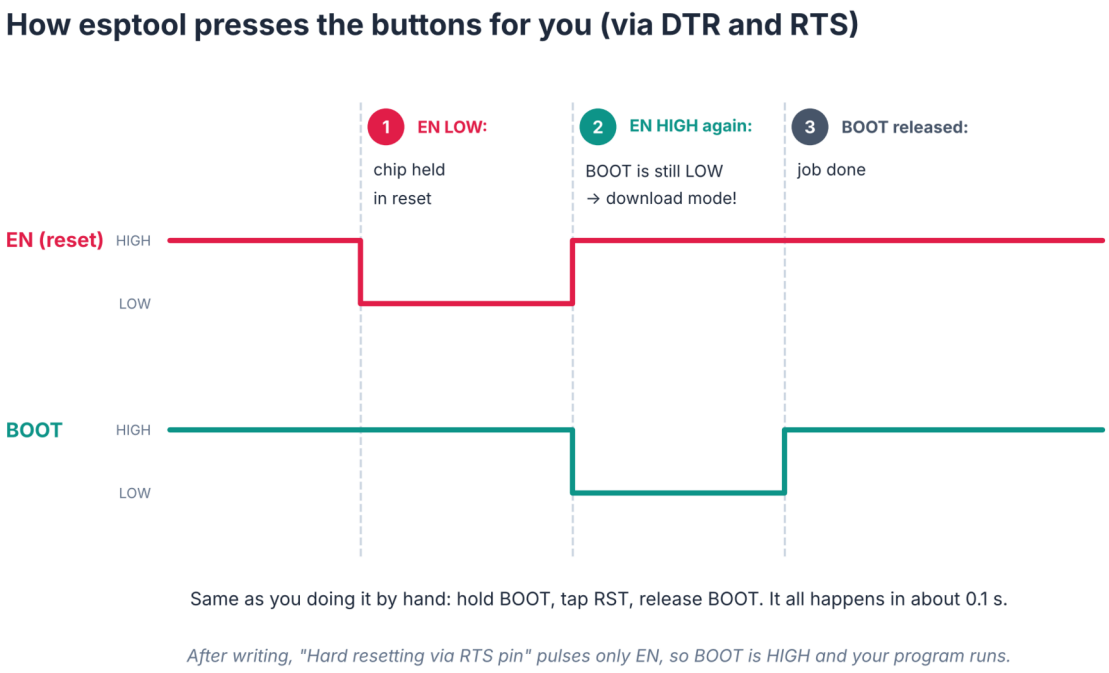


Figure 30. *The automatic reset sequence. Time runs left to right; HIGH means 3.3 volts, LOW means 0 volts.*

> **COMMON MISTAKE**
>
> If flashing hangs at `Connecting.....`, the automatic trick didn't work (some cables and USB hubs interfere). Do it by hand: **hold BOOT, press and release RST, then release BOOT**, and run the command again.

8d. What esptool does, step by step

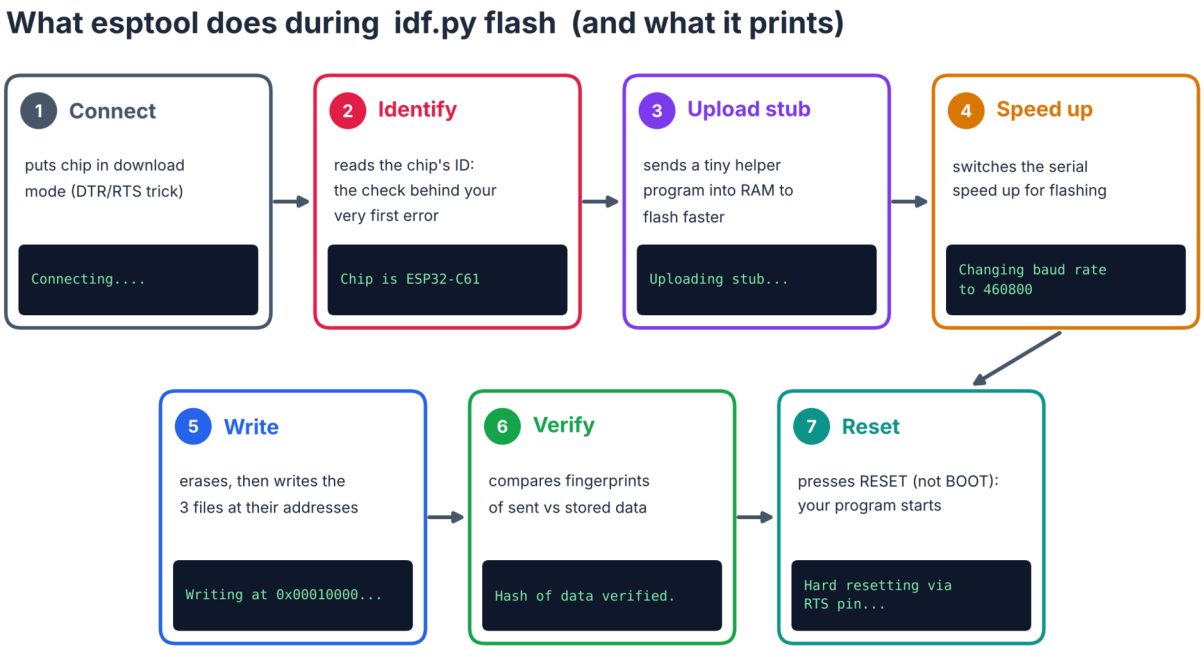


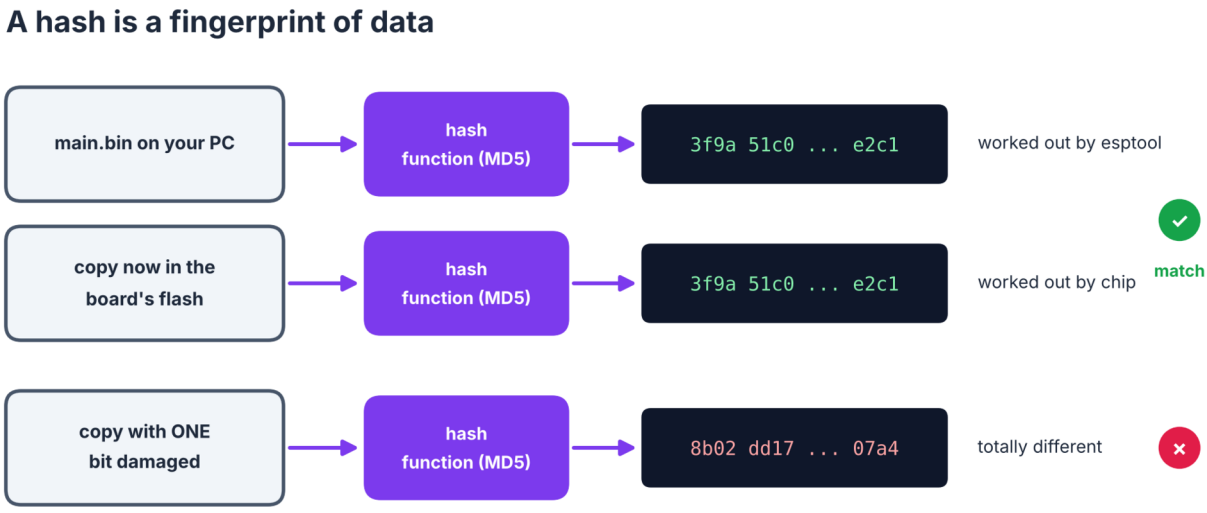
Figure 31. *esptool's seven stages, with the line it prints for each.*

```
YOU SHOULD SEE (similar to this)
Serial port COM19
Connecting....
Chip is ESP32-C61 (revision ...)
...
Uploading stub...
Running stub...
Stub running...
Changing baud rate to 460800
Changed.
...
Writing at 0x00010000... (25 %)
...
Hash of data verified.

Leaving...
Hard resetting via RTS pin...
```

Two lines deserve a closer look:

- **Chip is ESP32-C61**: esptool reads the identity number stored inside the chip. This is exactly the check that produced your very first Arduino IDE error. This time the target matches.
- **Hash of data verified**: a **hash** is a short fingerprint calculated from a block of data. esptool compares the fingerprint of what it sent with the fingerprint of what is now stored in flash. If they match, nothing was damaged on the way.



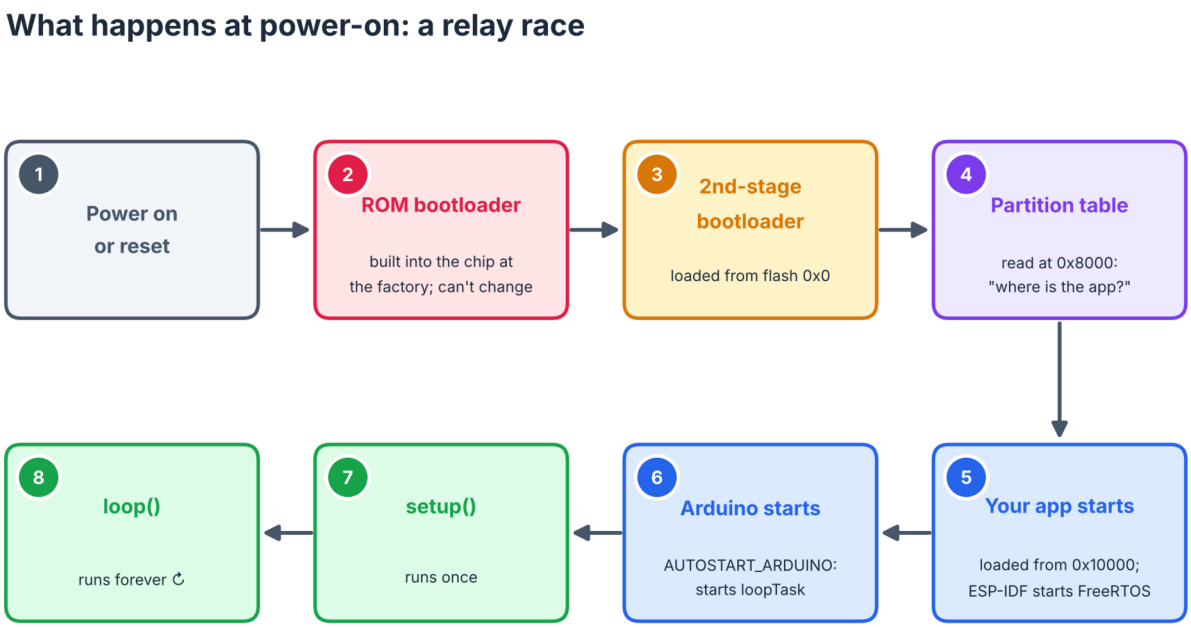
Comparing two short fingerprints is much faster than reading the whole file back. (Fingerprint values are illustrative.)

Figure 32. *Even one changed bit gives a completely different fingerprint, so a match proves the data arrived intact.*

The **stub** is a tiny helper program that esptool first sends into the chip's RAM. It knows tricks the ROM bootloader doesn't, such as talking faster (460800 baud instead of the starting 115200) and receiving compressed data, which makes flashing much quicker.

8e. The board starts your program

After `Hard resetting via RTS pin...`, esptool taps EN again, but this time leaves BOOT high, so the chip starts normally. A chain of programs now hands over to the next, like runners in a relay race:



Each runner hands the baton to the next. Steps 1–6 are the boot messages you see at the top of the monitor.

Figure 33. *The boot sequence, from power-on to your loop().*

The monitor shows this journey. First come messages from the boot chain, and then your program's output:

```
YOU SHOULD SEE (similar to this)
ESP-ROM:esp32c61-...
...
I (...) boot: ESP-IDF v5.5.5 2nd stage bootloader
...
I (...) boot: Partition Table:
I (...) boot: ## Label            Usage          Type ST Offset   Length
I (...) boot:  0 nvs              WiFi data        01 02 00009000 00006000
...
Hello world!
Hello world!
Hello world!
```

Lines starting with `ESP-ROM` come from the ROM bootloader. Lines starting with `I (...) boot:` are **I**nformation messages from the second-stage bootloader; the number in brackets is the time in milliseconds since start-up. Then your program begins, and **Hello world!** appears once every second.

8f. Why the text is readable: matching baud rates

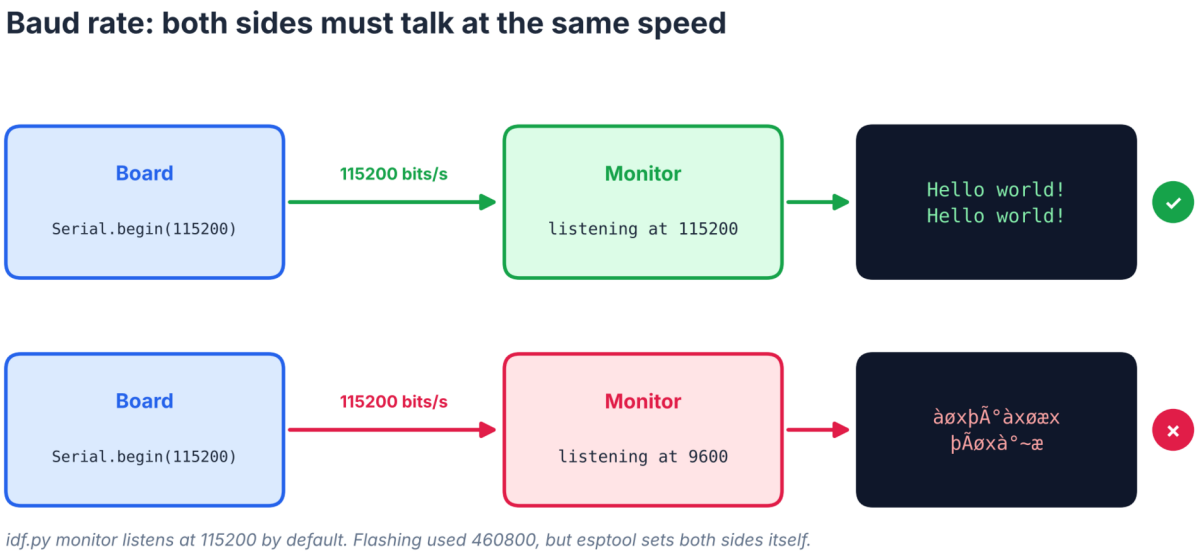


Figure 34. *If the monitor listens at a different speed from the one the board talks at, you get garbage characters.*

Your sketch called `Serial.begin(115200)`, and `idf.py monitor` listens at 115200 baud by default, so the text arrives clean. Flashing used a faster 460800 baud, but that doesn't clash: esptool switched both sides to that speed itself, and the monitor is a separate step afterwards.

> **YOU DID IT!**
> If you see **Hello world!** printing every second, your ESP32-C61 is running a program that you built from source with ESP-IDF and the Arduino core. You now understand every step between typing code and seeing it run, which is more than most people who just press Upload ever learn.

Changing your program from now on

Steps 1 to 5 were one-time setup for this project. From now on, the cycle is short:

1. Edit `main\main.cpp` (for example in Notepad or VS Code) and save it.
2. Run `idf.py -p COM19 flash monitor`. Because `flash` builds first whenever something has changed, this one command does build, flash and monitor.
3. Press **Ctrl +]** to leave the monitor when you are done.

> **TRY IT**
> Change the message to your own name and the delay to `500`. Flash again. How many times per second does it print now? How long did this build take compared with the first one, and why?

NEW WORDS IN THIS SECTION

Address: a position in memory, counted in bytes from the start.

Partition / partition table: a named region of flash / the map listing all regions.

ROM / ROM bootloader: permanent factory memory / the start-up program stored in it.

Second-stage bootloader: the bootloader in flash (from ESP-IDF) that loads your app.

Download mode: the ROM bootloader mode in which it waits for esptool's commands.

Strapping pin: a pin read only at start-up to choose a mode (BOOT is one).

EN pin: enable pin; pulling it low resets the chip.

DTR / RTS: two extra control lines on the USB-to-UART bridge, used by esptool to reset the chip.

Hash: a short fingerprint calculated from data, used to check it is unchanged.

Stub: a small helper program esptool loads into RAM to make flashing faster.

Serial monitor: a program that shows the text the board sends over the serial port.

CHECKPOINT

Without looking back: name the three files that were flashed and the address of each. What would happen if `main.bin` were written at `0x0` instead?

CHAPTER 6

Using this method for other ESP chips

Nothing in Chapter 5 was special to the ESP32-C61 except one word: the target name in Step 5. This chapter shows how to use the same method for any ESP chip, and in particular how to solve the real problem: **a board whose chip the Arduino IDE doesn't support yet**.

6.1 Why chips go missing from the Arduino IDE

When Espressif releases a new chip, support for it reaches the different tools in stages. ESP-IDF supports it first, because Espressif writes ESP-IDF together with the chip. Then the Arduino core's source code learns about it. Only at the very end does the Arduino IDE package ship the pre-compiled files for it. Between those stages there is a gap, sometimes many months long, and that gap is exactly where this manual's method helps.

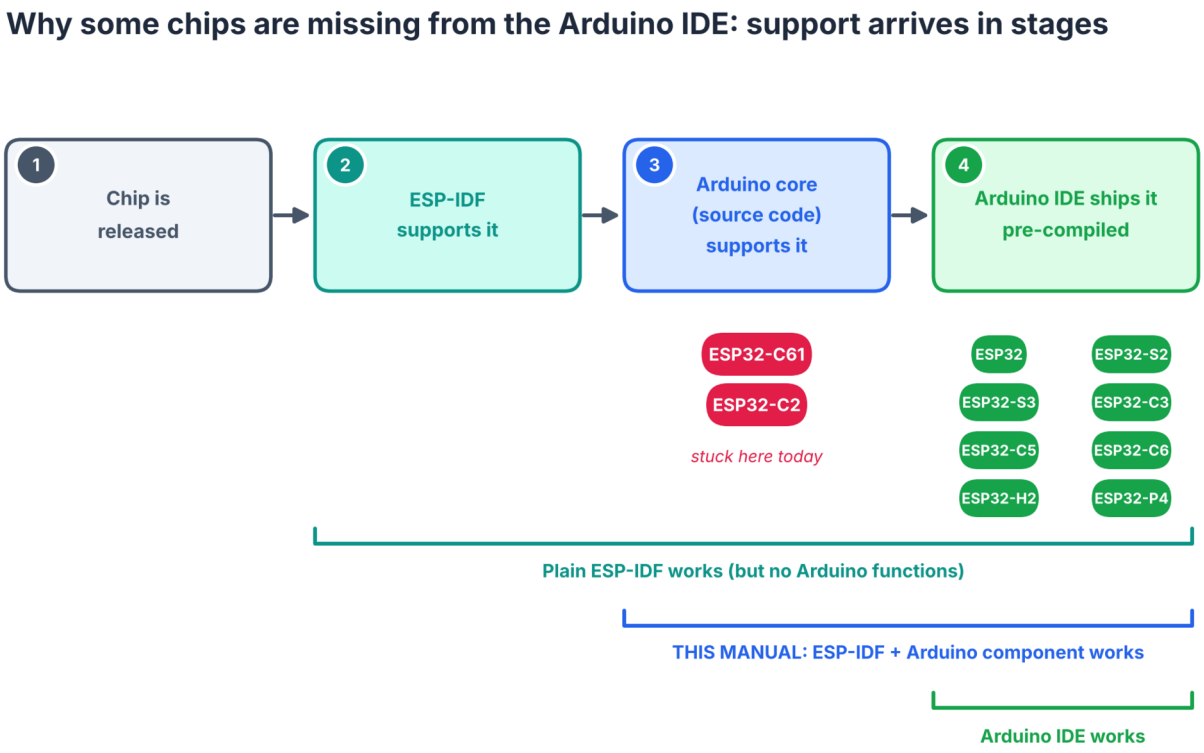


Figure 35. *Support arrives in four stages. The ESP-IDF + Arduino component method works as soon as stage 3 is reached; the Arduino IDE has to wait for stage 4.*

At the time of writing (Arduino core 3.3.12), the Arduino core's own documentation says the **ESP32-C2** and **ESP32-C61** are supported only "as an ESP-IDF component or by rebuilding the static libraries". Those are the two chips for which this manual is the practical answer today. Future chips will very likely go through the same gap.

6.2 Which route should I use?

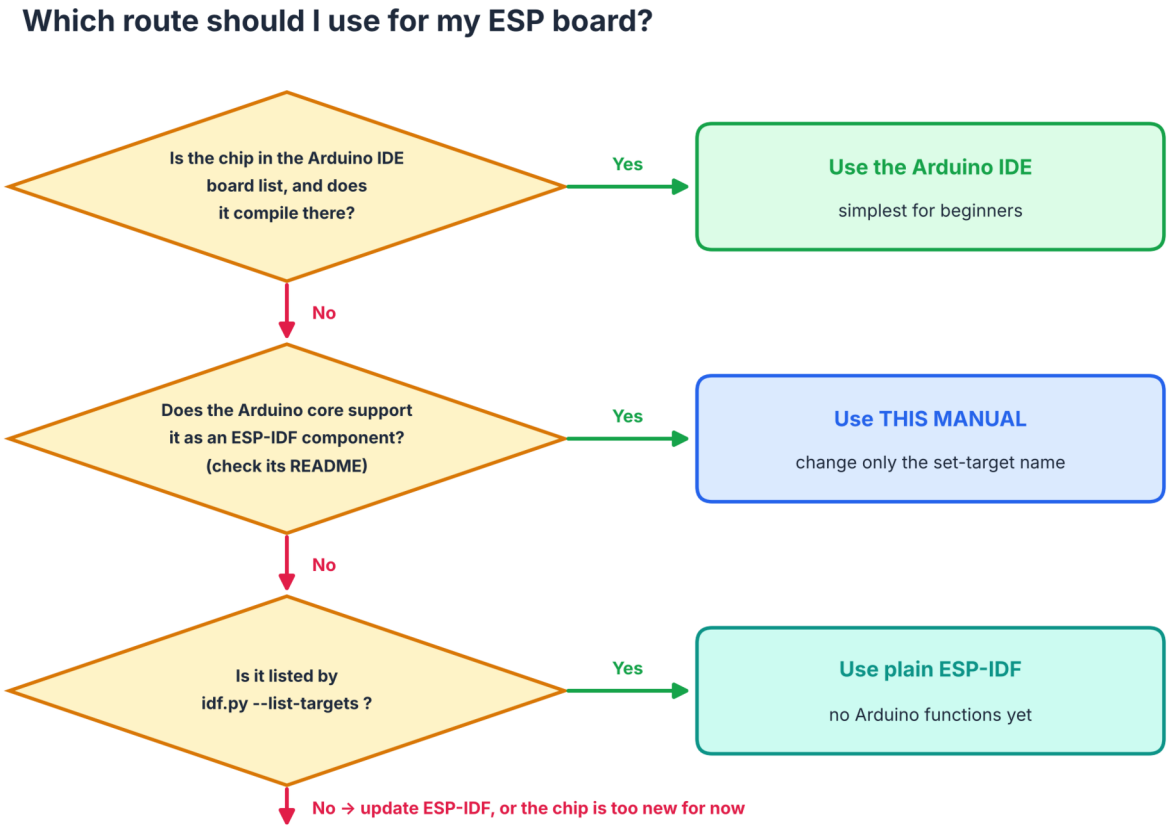


Figure 36. *Three questions decide the route for any ESP board.*

How to answer each question:

1. **Arduino IDE:** install the “esp32 by Espressif Systems” package, look under **Tools → Board → esp32**, and try compiling a blank sketch for your chip.
2. **Arduino core as a component:** open the arduino-esp32 project page on GitHub (github.com/espressif/arduino-esp32). Its README has a **Supported Chips** table, followed by a note listing the chips that need “Arduino as an ESP-IDF component”.
3. **ESP-IDF:** in the ESP-IDF terminal, run `idf.py --list-targets`. It prints every chip your installed ESP-IDF can build for.

```
TYPE THIS
idf.py --list-targets
```

> **NOTE**
> For chips the Arduino IDE already supports, the IDE is the easier choice for first-year students. The ESP-IDF route still works for them, and it is a good way to practise this lab if your college doesn’t have C61 or C2 boards.

6.3 The ESP chip family at a glance

Use this table to find your chip's target name for Step 5 and what to expect from it. Values come from Espressif's datasheets and the arduino-esp32 README; always check your own board's documentation too.

Chip	set-target name	Processor	Wireless	USB on the chip	Boot-loader at	Arduino IDE?
ESP32	`esp32`	Xtensa, 2 cores, 240 MHz	Wi-Fi 4, Bluetooth Classic + LE	none (needs a bridge)	0x1000	Yes
ESP32-S2	`esp32s2`	Xtensa, 1 core, 240 MHz	Wi-Fi 4, no Bluetooth	USB OTG	0x1000	Yes
ESP32-S3	`esp32s3`	Xtensa, 2 cores, 240 MHz	Wi-Fi 4, Bluetooth LE 5	USB OTG + USB Serial/JTAG	0x0	Yes
ESP32-C2	`esp32c2`	RISC-V, 1 core, 120 MHz	Wi-Fi 4, Bluetooth LE 5	none (needs a bridge)	0x0	**No: use this manual**
ESP32-C3	`esp32c3`	RISC-V, 1 core, 160 MHz	Wi-Fi 4, Bluetooth LE 5	USB Serial/JTAG	0x0	Yes
ESP32-C5	`esp32c5`	RISC-V, 1 core, 240 MHz	Wi-Fi 6 (2.4 + 5 GHz), LE 5, 802.15.4	USB Serial/JTAG	0x2000	Yes
ESP32-C6	`esp32c6`	RISC-V, 1 core, 160 MHz	Wi-Fi 6, LE 5, 802.15.4	USB Serial/JTAG	0x0	Yes
ESP32-C61	`esp32c61`	RISC-V, 1 core, 160 MHz	Wi-Fi 6, Bluetooth LE 5	USB Serial/JTAG	0x0	**No: use this manual**
ESP32-H2	`esp32h2`	RISC-V, 1 core, 96 MHz	no Wi-Fi; LE 5, 802.15.4	USB Serial/JTAG	0x0	Yes
ESP32-P4	`esp32p4`	RISC-V, 2 cores, up to 400 MHz	none built in	USB OTG + USB Serial/JTAG	0x2000	Yes

802.15.4 is the low-power radio used by Thread and Zigbee smart-home devices. *USB Serial/JTAG* and *USB OTG* are two kinds of USB built into the chip itself; see 6.5.

6.4 What changes when you switch chips

Here is every step of the lab, with what (if anything) changes for a different chip:

Lab step	What changes for another chip
1. Check version	Nothing, as long as your ESP-IDF supports the chip (`idf.py --list-targets`).
2–4. Create, explore, pin	Nothing. The hello_world example is not tied to any chip.
5. Set target	**Use your chip's target name** from the table in 6.3.
6. Build	Nothing. (The first build for each new target is slow again.)
7. Find the port	The port name depends on the board's USB type: a bridge chip or native USB (see the USB figure in Step 7).
8. Flash and monitor	Nothing to type differently: idf.py uses the right bootloader address for the chip automatically. Native-USB boards may need manual download mode the first time.
Your own code	Check pin numbers, and that the chip has the features you use (Wi-Fi, classic Bluetooth...).

Worked example: an ESP32-C2 board

Here is the complete lab for an ESP32-C2 board in one block. Compare it with Chapter 5: only the folder name and the target name are different.

TYPE THIS

```
cd D:\esp\projects
idf.py create-project-from-example "espressif/arduino-esp32^3.3.12:hello_world"
ren hello_world hello_c2
cd hello_c2
notepad main\idf_component.yml     # pin version: '^3.3.12' as in Step 4
idf.py set-target esp32c2
idf.py build
idf.py -p COM7 flash monitor          # use your own COM number
```

Everything after a `#` is a *comment*: PowerShell ignores it, so it is safe to type or leave out. (If you use the CMD version of the ESP-IDF terminal, leave the comments out.) `ren` renames a folder. Rename the project *before* `set-target`, because the build folder remembers the full path of the project.

> COMMON MISTAKE
>
> **ESP32-C2 shows garbage text in the monitor.** Many ESP32-C2 boards use a **26 MHz crystal** (the tiny component that sets the chip's clock), while ESP-IDF assumes 40 MHz. All the timing is then wrong, including the baud rate. Fix it once per project:
>
> 1. Run `idf.py menuconfig`.
> 2. Go to **Component config → Hardware Settings → Main XTAL Config → Main XTAL frequency** and choose **26 MHz**.
> 3. Save, exit, and run `idf.py -p COMx flash monitor` again.

6.5 Boards with native USB: getting Serial on the USB port

Many newer boards have no CP210x or CH340 bridge. The USB connector is wired straight to the ESP chip, which has USB built in (see the USB figure in Step 7). Flashing works the same way: esptool talks to the chip's own USB and resets it itself. But there is one surprise: by default, `Serial` in the Arduino core still sends its text to the chip's **UART pins**, not to the USB port, so the monitor stays empty.

Espressif provides a ready-made example that sends `Serial` to the USB port instead. Create your project from it rather than from hello_world:

TYPE THIS

```
idf.py create-project-from-example "espressif/arduino-esp32=3.3.12:hw_cdc_hello_world"
```

Here `=3.3.12` means "exactly version 3.3.12". The example's top-level `CMakeLists.txt` adds two settings, `ARDUINO_USB_CDC_ON_BOOT=1` and `ARDUINO_USB_MODE=1`, which tell the Arduino core to connect `Serial` to the chip's USB Serial/JTAG port. The rest of the lab is unchanged: pin the version, set the target, build, flash.

> NOTE
> The example's documentation lists the ESP32-C3, C6, H2 and S3. Other chips with USB Serial/JTAG (such as the C5, C61 and P4) use the same settings, but test it on your own board before relying on it in class. If your board has a separate UART port with a bridge chip, the simplest option is to use that port instead.

> COMMON MISTAKE
> **A native-USB board doesn't connect at all**, or disappears from Device Manager after flashing a broken program: put it in download mode by hand. Hold **BOOT**, press and release **RST** (or plug the USB cable in while holding BOOT), release BOOT, and flash again.

6.6 Future chips: how to check versions yourself

New ESP chips keep arriving. When one isn't in the Arduino IDE yet, you can work out the right setup yourself, the same way this manual was made for the C61:

1. **Is it in the Arduino core?** Check the arduino-esp32 README (6.2). If it's listed as "requires Arduino as an ESP-IDF component", this method will work.
2. **Which ESP-IDF version does that Arduino core need?** The "Arduino as an ESP-IDF component" page of the Arduino-ESP32 documentation states it. For core 3.3.12 it says ESP-IDF v5.5.
3. **Does that ESP-IDF version build for the chip?** Install it and run `idf.py --list-targets`.
4. **Pin both versions**: install the matching ESP-IDF (Chapter 3), and set the Arduino core version in `idf_component.yml` (Step 4).

FUN FACT

ESP-IDF sometimes supports a brand-new chip as a *preview* target before it is finished. Preview targets are hidden from the normal list; `idf.py --preview set-target <chip>` uses one. Preview support is experimental, so expect rough edges, and never use it for a class until it becomes a normal target.

6.7 When this method can't help

- **The Arduino core doesn't support the chip at all yet** (stage 2 in the figure in 6.1): you can still program it with plain ESP-IDF, using ESP-IDF's own functions instead of Arduino ones.
- **The board isn't an Espressif chip** (Arduino Uno and Nano, STM32, Raspberry Pi Pico...): ESP-IDF can't build for these at all. They have their own board packages, usually well supported in the Arduino IDE.

NEW WORDS IN THIS SECTION

Native USB: USB built into the ESP chip itself, so the board needs no separate bridge chip.

USB Serial/JTAG: a simple built-in USB port on newer ESP chips that acts as a serial port (and a debugging port).

Crystal (XTAL): a small component that ticks at an exact frequency and sets the chip's clock.

Preview target: a chip that ESP-IDF supports only experimentally so far.

802.15.4: a low-power radio standard used by Thread and Zigbee devices.

CHECKPOINT

A classmate has an ESP32-C2 board and an ESP32-C3 board. Which one needs this manual's method, and which word changes in Step 5 for it?

CHAPTER 7

Troubleshooting

Errors are normal. Professional engineers see them every day; the skill is reading them calmly. Start by finding the *first* red error line in the output (later errors are often caused by the first one), then match it below.

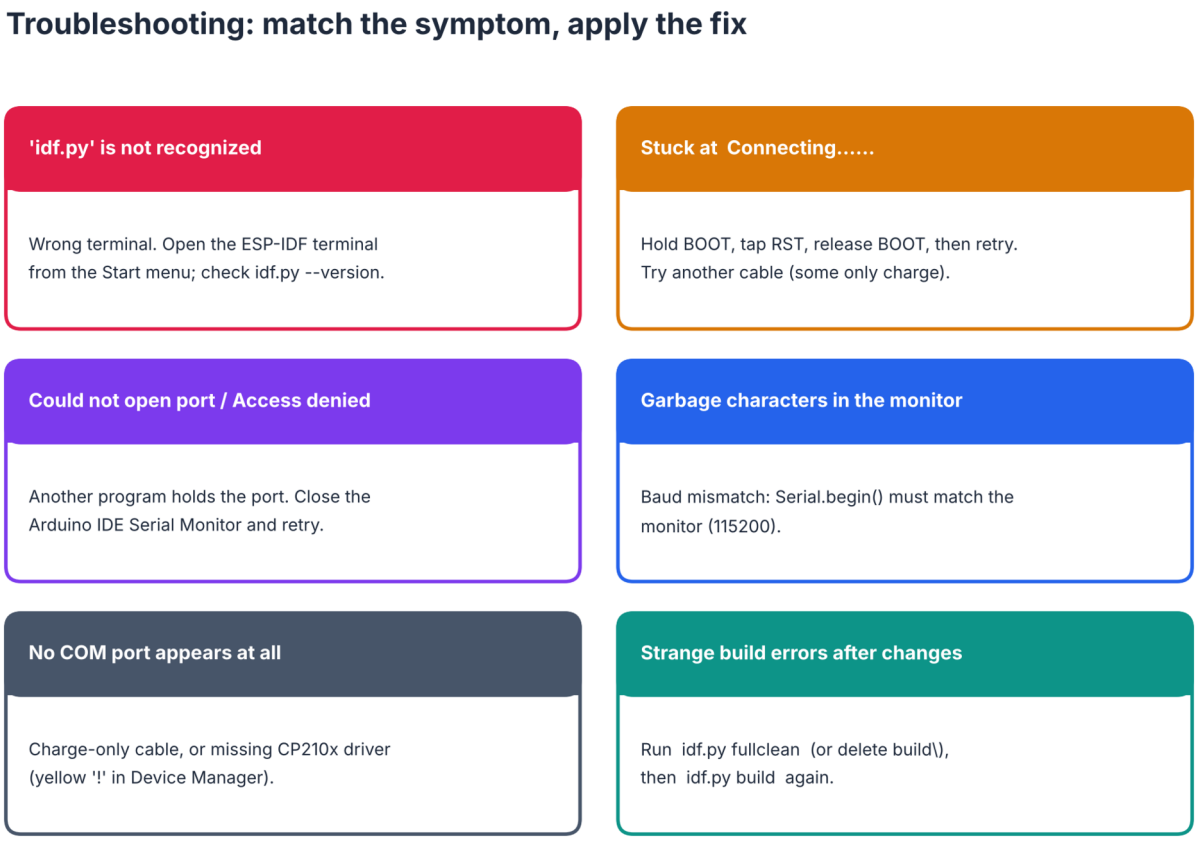


Figure 37. *The six problems students meet most often, and their fixes.*

Symptom	Likely cause	Fix
`'idf.py' is not recognized`	Ordinary terminal; ESP-IDF not in PATH.	Open the ESP-IDF terminal from the Start menu (Step 1).
`idf.py --version` shows v6.x or v5.4	Wrong ESP-IDF installed or wrong shortcut opened.	Install v5.5.5 (Chapter 3) and open its own shortcut.
Download fails in Step 2 or 5	No internet, or a firewall/proxy blocking it.	Check the connection; on college networks ask the lab in-charge about proxy settings.
YAML error mentioning `idf_component.yml`	Wrong indentation or a Tab character.	Retype the 3 lines with 2 and 4 spaces (Step 4).
Build errors that mention a path with spaces	Project or ESP-IDF in a folder with spaces.	Move the project to `D:\esp\projects` (Chapter 3.4).

Symptom	Likely cause	Fix
Strange errors after changing settings or versions	Old files left in `build\`.	Run `idf.py fullclean`, then `idf.py build`.
Stuck at `Connecting.....`	Auto-reset didn't work.	Hold BOOT, tap RST, release BOOT; retry. Try another cable or USB socket.
`could not open port` / `Access is denied`	Another program is using the COM port.	Close the Arduino IDE Serial Monitor and other serial programs.
`This chip is ESP32-..., not ESP32-C61`	Target doesn't match the board.	`idf.py set-target esp32c61` (for a C61 board), then build again.
Garbage characters in the monitor	Baud rate mismatch.	`Serial.begin()` must use 115200 to match the monitor.
Monitor shows nothing	Wrong port, or board waiting after flashing.	Press RST on the board; check the COM number in Device Manager.
Garbage text on an ESP32-C2 board	Board uses a 26 MHz crystal.	menuconfig → Hardware Settings → Main XTAL frequency → 26 MHz (Chapter 6.4).
Native-USB board: flashing works but monitor is empty	`Serial` still goes to the UART pins.	Start from the hw_cdc_hello_world example (Chapter 6.5).
Native-USB board won't connect	Running program blocks the USB port.	Hold BOOT, tap RST (or plug in while holding BOOT), release, flash again.
No COM port in Device Manager	Charge-only cable or missing driver.	Try a data cable; install the CP210x driver.

> **NOTE**
>
> `idf.py fullclean` deletes the build folder's contents so the next build starts fresh. It never touches your `main.cpp`. The next build will be slow again, like the first one.

CHAPTER 8

Quick reference

Commands

Command	What it does	When
`idf.py --version`	Shows the ESP-IDF version; proves you're in the right terminal.	Start of every session
`idf.py --list-targets`	Lists every chip this ESP-IDF can build for.	Choosing a target
`idf.py create-project-from-example "espressif/arduino-esp32^3.3.12:hello_world"`	Creates a new Arduino-ready project.	New project
`idf.py set-target esp32c61`	Chooses the chip; creates sdkconfig; downloads components.	Once per project
`idf.py menuconfig`	Opens the settings menu (edits sdkconfig).	Changing settings
`idf.py build`	Compiles everything.	After each change
`idf.py -p COM19 flash`	Builds if needed, then writes to the board.	After each change
`idf.py -p COM19 monitor`	Shows serial output. Exit with Ctrl +].	Watching output
`idf.py -p COM19 flash monitor`	Both of the above in one go.	Most of the time
`idf.py size`	Shows how much flash and RAM your program uses.	Curiosity / big projects
`idf.py partition-table`	Prints the partition table.	Curiosity
`idf.py fullclean`	Deletes all build output for a fresh start.	Strange build errors

Starting a new project: checklist

1. Open the ESP-IDF terminal; `idf.py --version` shows v5.5.5.
2. `cd D:\esp\projects` and run `create-project-from-example` (you can rename the created `hello_world` folder before going on, e.g. `ren hello_world blink_led`; the new name must not contain spaces).
3. Edit `main\idf_component.yml` to pin `version: '^3.3.12'`.
4. `idf.py set-target esp32c61` (or your chip's name from Chapter 6.3).
5. Put your code in `main\main.cpp` (keep `#include "Arduino.h"` at the top).
6. `idf.py -p COMx flash monitor`.

Files at a glance

File or folder	Edit it?
`main\main.cpp`	Yes: your code.
`main\idf_component.yml`	Yes: when adding components or pinning versions.
`sdkconfig.defaults`	Yes, but only affects a fresh `sdkconfig` (delete `sdkconfig` to apply).
`sdkconfig`	Only through `idf.py menuconfig`.
`CMakeLists.txt` (both)	Rarely.
`dependencies.lock`	No, but keep it with the project.
`managed_components\`	Never.
`build\`	Never; safe to delete.

CHAPTER 9

Glossary

Every new word from this manual, in alphabetical order.

Word	Meaning
Address	A position in memory, counted in bytes from the start, usually written in hex (e.g. 0x10000).
Argument	A word after a program's name in a command, telling it what to do.
Arduino core	The code that makes Arduino functions (Serial, delay, digitalWrite...) work on one chip family.
Baud rate	The speed of a serial connection in bits per second. Both ends must use the same value.
Binary (.bin)	A file of raw machine code and data, ready to write into flash.
BOOT pin	A strapping pin; held low at start-up, it puts the chip into download mode.
Bootloader	A small program that starts the main program. The ROM bootloader is built into the chip; the second-stage bootloader is in flash.
Build system	Software that decides what to compile and in what order, then runs the compiler (CMake and Ninja here).
Byte	8 bits; one small number from 0 to 255. 1 KB = 1024 bytes, 1 MB = 1024 KB.
CMake	The tool that plans an ESP-IDF build (the configure stage).
COM port	The name Windows gives a serial connection, such as COM19.
Comment	A note in a file for humans that the computer ignores (lines starting with # in sdkconfig and YAML).
Compiler	A program that translates source code into machine code.
Component	A self-contained package of code in ESP-IDF. Your main folder is one; the Arduino core is another.
Component Manager	The ESP-IDF tool that downloads components from the ESP Component Registry.
Configure stage	The planning step before compiling, run by CMake.
CH340	A WCH USB-to-UART bridge chip; does the same job as a CP210x.
CP210x	A Silicon Labs USB-to-UART bridge chip used on many boards.
Crystal (XTAL)	A small component that ticks at an exact frequency and sets the chip's clock (usually 40 MHz; 26 MHz on many ESP32-C2 boards).
Dependency	Something your code needs that it doesn't contain itself.

Word	Meaning
dependencies.lock	File recording the exact component versions used, so builds can be repeated.
Download mode	ROM bootloader mode in which the chip waits for esptool commands.
Driver	Software that lets Windows (or a program) operate a piece of hardware.
DTR / RTS	Two control lines of the USB-to-UART bridge; esptool uses them to reset the chip automatically.
ELF (.elf)	The linked program plus extra information such as function names.
EN pin	Enable pin; pulling it low resets the chip.
ESP-IDF	Espressif IoT Development Framework: Espressif's official framework for ESP32 chips.
esptool	Espressif's program for identifying, flashing and resetting chips over serial.
Flash memory	Memory that keeps its contents with the power off; holds your program.
Framework	A large set of ready-made code and tools to build programs on.
FreeRTOS	A small operating system that shares the processor between tasks.
GPIO	General-Purpose Input/Output: a pin your program can switch or read.
Hash	A short fingerprint calculated from data; used to check the data is unchanged.
Header file	A file (.h) that declares functions so other code can use them.
Hexadecimal (hex)	Base-16 numbers using 0–9 and a–f, written with 0x in front.
idf.py	ESP-IDF's main command; it calls the right tool for each job.
Incremental build	A build that only recompiles the files that changed.
Indentation	Spaces at the start of a line; YAML uses it to show structure.
Linker	The tool that joins object files and fills in the addresses between them.
Machine code	Instructions as numbers; the only language a processor understands.
Native USB	USB built into the ESP chip itself, so the board needs no bridge chip.
Microcontroller	A small complete computer on one chip, made to control hardware.
Ninja	The tool that carries out the build plan quickly, job by job.
Object file (.o)	Machine code for one source file, with gaps for things defined elsewhere.
Operating system	Software that manages the hardware and shares the processor between tasks.
Partition / partition table	A named region of flash / the map listing where each region is.
PATH	The list of folders Windows searches when you type a command.

Word	Meaning
Port	A connection point; here, the serial connection to the board.
Preview target	A chip that ESP-IDF supports only experimentally so far.
Pre-compiled	Already translated into machine code ahead of time.
Pre-release	An unfinished test version of software (alpha, beta, rc).
Project	A folder with your code and the files telling ESP-IDF how to build it.
Prompt	The text at the start of a terminal line showing it is ready for a command.
Python	A programming language; idf.py and esptool are written in it.
RAM	Fast working memory, cleared at power-off.
Reproducible build	A build that always produces the same result from the same project.
RISC-V	An open processor design used by the ESP32-C61 (pronounced “risk-five”).
ROM	Read-only memory, written at the factory and never changed.
sdkconfig	The file holding every ESP-IDF setting for a project.
Semantic versioning	MAJOR.MINOR.PATCH version numbers; a major change may break old code.
Serial monitor	A program that shows the text a board sends over the serial port.
Source code	Human-readable code written by people, before compiling.
Strapping pin	A pin read only at start-up to choose a mode.
Stub	A small helper program esptool loads into RAM to make flashing faster.
Target	The chip you are building for (esp32c61 here).
Task	One job scheduled by FreeRTOS.
Terminal	A text window where you type commands.
Tick	FreeRTOS’s regular timing heartbeat (1000 per second in our projects).
Toolchain	The set of tools (compiler, linker, helpers) that turns code into a binary.
TX / RX	The transmit and receive wires of a UART connection.
USB Serial/JTAG	A simple USB port built into newer ESP chips that works as a serial port and a debug port.
UART	A simple way of sending data one bit after another on a wire.
USB-to-UART bridge	A chip that converts USB from the PC into UART for the microcontroller.
Virtual environment (venv)	A private copy of Python and its packages used only by ESP-IDF.
Xtensa	The processor design used by the original ESP32, ESP32-S2 and ESP32-S3.

Word	Meaning
YAML (.yml)	A simple settings format that uses indentation to show structure.

CHAPTER 10

Check your understanding

Try to answer each question in your own words before looking at the answers on the next page. If you get stuck, the chapter or step where the idea is explained is a good place to re-read.

1. Name the two jobs hidden behind the Arduino IDE's Upload button.
2. Why couldn't the Arduino IDE build a program for the ESP32-C61?
3. Why does Route B (ESP-IDF with the Arduino component) work?
4. A classmate types idf.py build and gets "not recognized". What is wrong?
5. Which versions does ^3.3.12 allow? Would it accept 3.4.1? 4.0.0?
6. Why did we replace version: '*' and pre_release: true?
7. Which folder must you never edit, and why?
8. You edited sdkconfig.defaults after the first build, but nothing changed. Why?
9. Why does the ESP32-C61 need a different compiler from the original ESP32?
10. What does the linker do?
11. Convert 0x8000 to an ordinary number.
12. List the three files flashed and their addresses.
13. What decides whether the chip runs your program or waits for esptool?
14. What does "Hash of data verified" prove?
15. The monitor shows random symbols instead of text. What do you check first?
16. Your friend has an ESP32-C2 board, which the Arduino IDE can't program. What do they change in this lab?
17. A board has no CP210x or CH340 chip. What is different about it?
18. Why is the second build much faster than the first?

> NOTE
>
> The Checkpoint questions inside the chapters:
>
> **Chapter 1**: compiling only makes the binary on your computer. Nothing reaches the board until it is flashed.
>
> **Chapter 2**: because ESP-IDF supports the C61 and compiles everything from source, so it doesn't need the missing pre-compiled files.
>
> **Chapter 6**: the ESP32-C2 needs this manual's method (the C3 works in the Arduino IDE). In Step 5 it changes `esp32c61` to `esp32c2`.
>
> **Step 8**: bootloader.bin at 0x0, partition-table.bin at 0x8000, main.bin at 0x10000. If main.bin were written at 0x0, it would overwrite the bootloader. The ROM bootloader would try to start your app as if it were the bootloader, and the board would not boot until you flashed it correctly again.

Answers

1. Compiling (turning code into a binary) and flashing (writing the binary into the board's flash memory).
2. Its ESP32 package needs ESP-IDF pre-compiled for each chip, and the pre-compiled ESP32-C61 files are not shipped yet. (The board entry is hidden in boards.txt for the same reason.)
3. ESP-IDF supports the C61 and compiles everything, including the Arduino core, from source for that chip, so nothing pre-compiled is needed.
4. They are in an ordinary terminal. Only the ESP-IDF terminal adds ESP-IDF's folders to PATH.
5. 3.3.12 and any newer 3.x version. 3.4.1: yes. 4.0.0: no.
6. They allow any version, even unfinished ones, which could break on ESP-IDF v5.5.5 and makes builds differ from day to day. Pinning gives reproducible builds.
7. managed_components, because the Component Manager controls it and can overwrite your changes.
8. sdkconfig already existed, and once it exists it wins. Use idf.py menuconfig, or delete sdkconfig so it is rebuilt from the defaults.
9. The C61 uses a RISC-V processor and the original ESP32 uses Xtensa; their machine code is completely different.
10. It joins all the object files into one program and fills in the gaps, connecting each call to the address where that function really lives.
11. 8 × 4096 = 32,768 bytes, which is exactly 32 KB.
12. bootloader.bin at 0x0, partition-table.bin at 0x8000, main.bin at 0x10000.
13. The BOOT strapping pin at the moment of start-up: held low means download mode, otherwise normal boot.
14. That the fingerprint of the data stored in flash matches the fingerprint of what esptool sent, so nothing was damaged.
15. The baud rate: Serial.begin() must match the monitor's speed (115200).
16. Only the target name in Step 5: idf.py set-target esp32c2. (On many C2 boards they must also set the crystal to 26 MHz in menuconfig.)
17. It uses the ESP chip's native USB. It shows up under a different name, esptool resets it through USB, and Serial needs routing to the USB port to appear in the monitor.
18. It is an incremental build: Ninja only redoes the jobs whose files changed (main.cpp, link, convert), not all 1576.

CHAPTER 11

What's next

You now have a working pipeline: code → build → flash → run. In the next lab we will bring existing Arduino sketches and libraries into this pipeline. Here is a preview of what changes:

- **#include "Arduino.h"** at the top of every file, because ESP-IDF doesn't add it silently like the Arduino IDE does.
- **Function prototypes**: the Arduino IDE also quietly declares your own functions for you. In a normal `.cpp` file you may need to declare them above `setup()` yourself.
- **Libraries become components**: libraries you would install with the Arduino Library Manager go into a `components` folder in the project, each with its own small `CMakeLists.txt`.
- **Check pin numbers** against your ESP32-C61 board's pin diagram; many examples are written for the original ESP32.
- **Bluetooth**: the C61 supports Bluetooth Low Energy only, so sketches using classic `BluetoothSerial` must be rewritten with the BLE library.

> FUN FACT
>
> Everything you learned here (targets, components, partitions, bootloaders, esptool) is exactly how professional ESP32 products are built. You have already used the same tools as the engineers who design them.

Versions used in this manual: ESP-IDF v5.5.5, arduino-esp32 3.3.12, esptool v4.12.0. Tested on an ESP32-C61 board. Written October 2026.

Prepared by Pragyan Sen Deka , Mtech Hybrid Student , Robotics and AI , IITG , Associate IIM Calcutta Innovation Park Technology and Innovation Council

This is an independent guide. It is not an official publication of IIT Guwahati or IIMCIP.